

RESOLUCIÓN DE PROBLEMAS

Búsqueda

FACULTAD POLITÉCNICA
INGENIERÍA EN ELECTRÓNICA

Inteligencia artificial - 9º Semestre

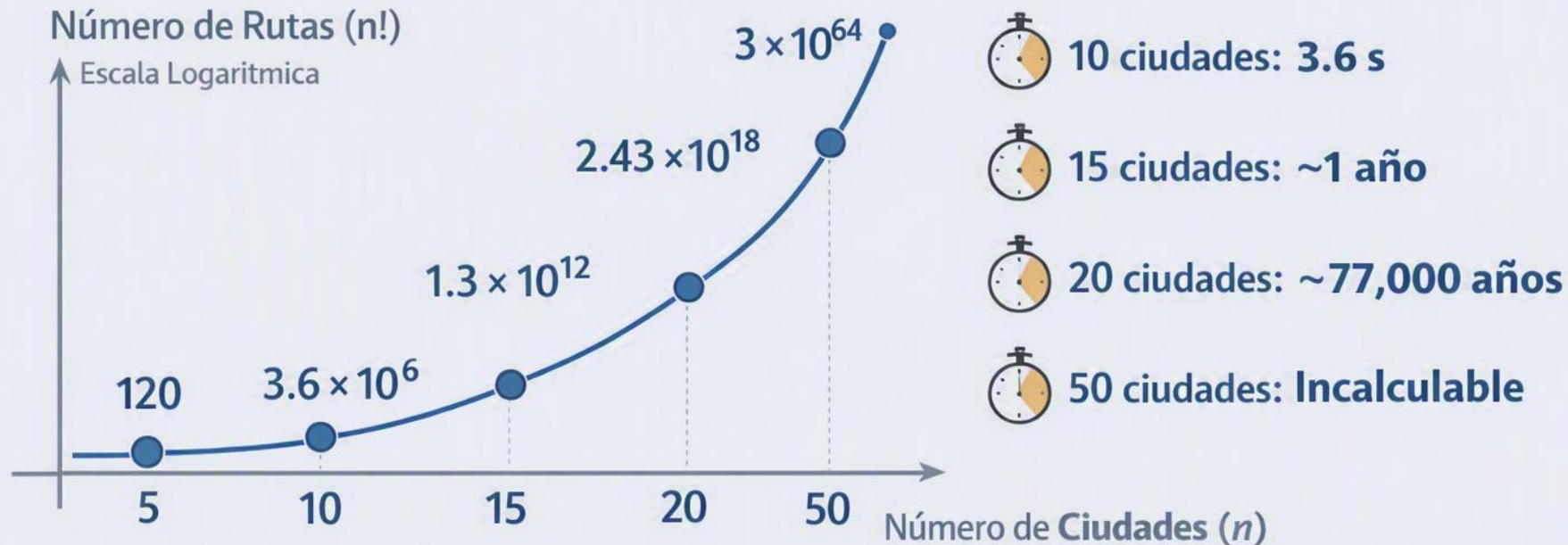
Prof. Hugo Suzuki

OBJETIVO

Desarrollar la capacidad de formular y resolver problemas en espacios de estados mediante estrategias de búsqueda, comprendiendo cómo un agente puede planificar secuencias de acciones y diseñar algoritmos de planificación de trayectorias en entornos con obstáculos, evaluando su efectividad y aplicabilidad.

Problema del Viajante de Comercio: Crecimiento Factorial y Tiempo de Procesamiento

Made with AI



Complejidad: $O(n!)$
Crecimiento Explosivo

$n > 15$
Métodos Clásicos Impracticables

Heurísticas y Metaheurísticas



RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA



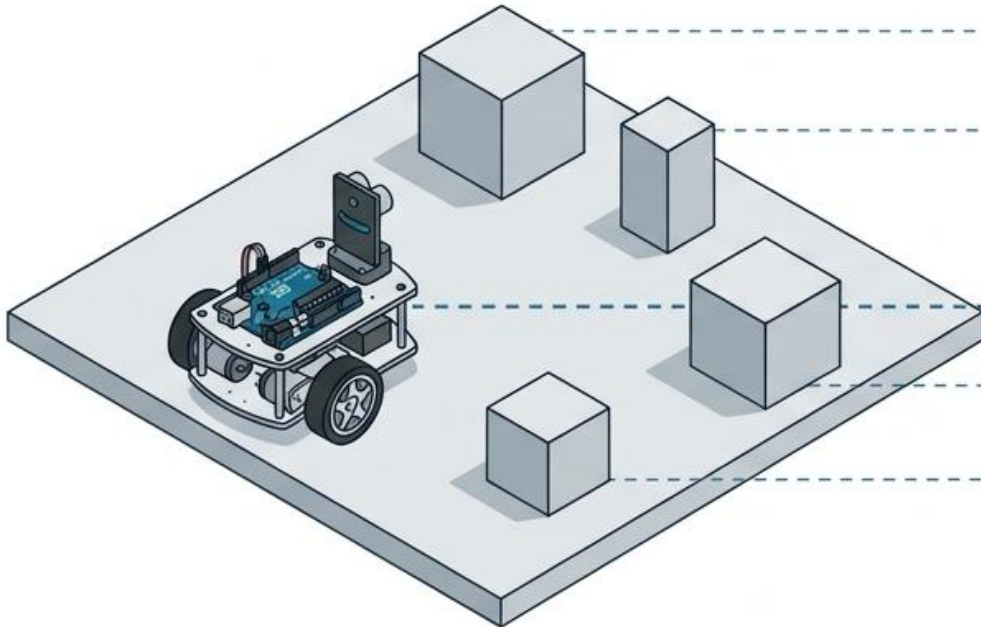
El Problema del Ingeniero: La Traducción del Entorno

Para que un robot navegue de un punto A a un punto B, primero debemos traducir un entorno físico continuo y complejo a una estructura de datos discreta que un algoritmo pueda procesar y resolver.

El Mundo Físico

Entorno Continuo

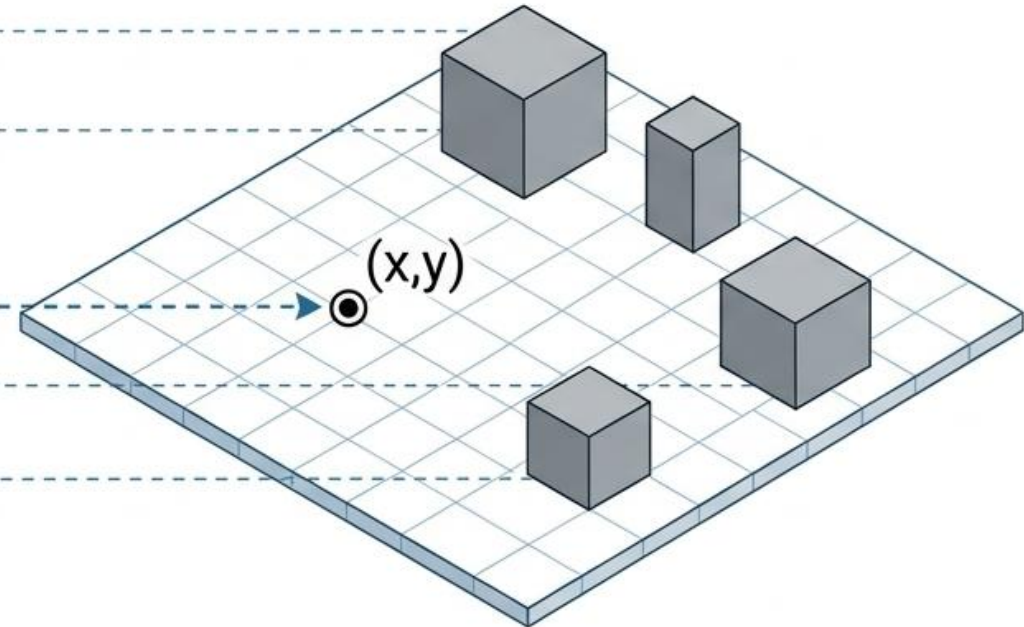
Infinitas posiciones posibles.



El Mundo Discreto

Cuadrícula de Navegación (Espacio de Estados)

Coordenadas discretas finitas.

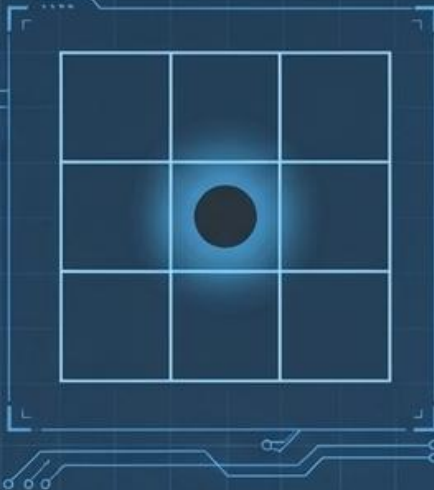


Formalización Matemática: El Espacio de Estados

S_0

Estado Inicial

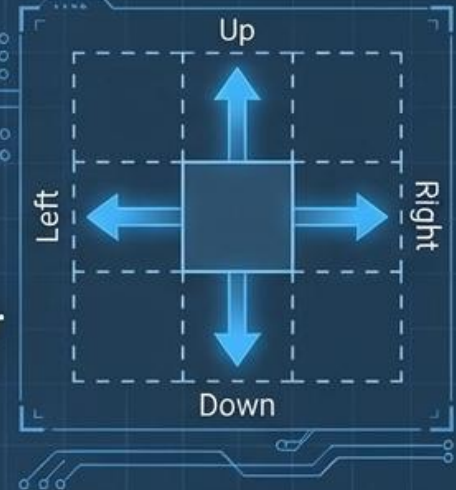
Dónde comienza el agente.
Ej: $En(x_0, y_0)$.



$A(s)$

Acciones y Función Sucesor

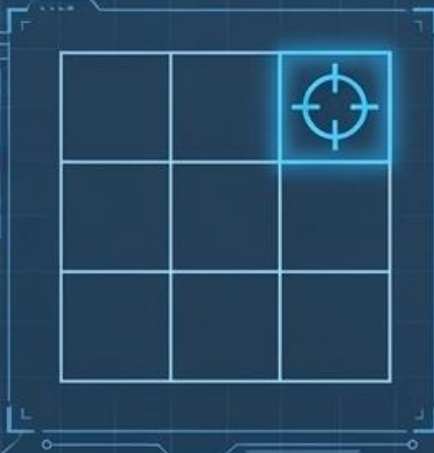
Transiciones legales desde un estado actual.



$Goal(s) == True$

Test Objetivo

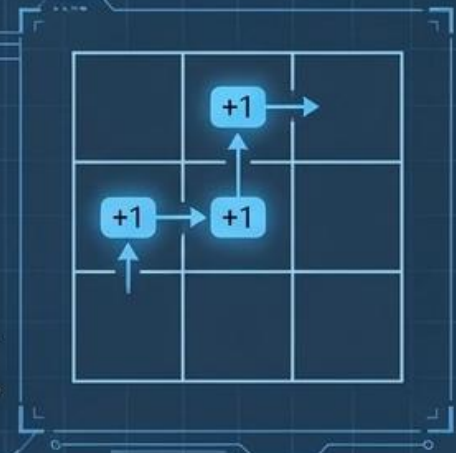
Condición lógica que verifica si la coordenada actual es la meta.



$c(s, a, s')$

Costo del Camino

Penalización numérica por ejecutar una acción. En cuadrículas simples, cada paso cuesta 1.

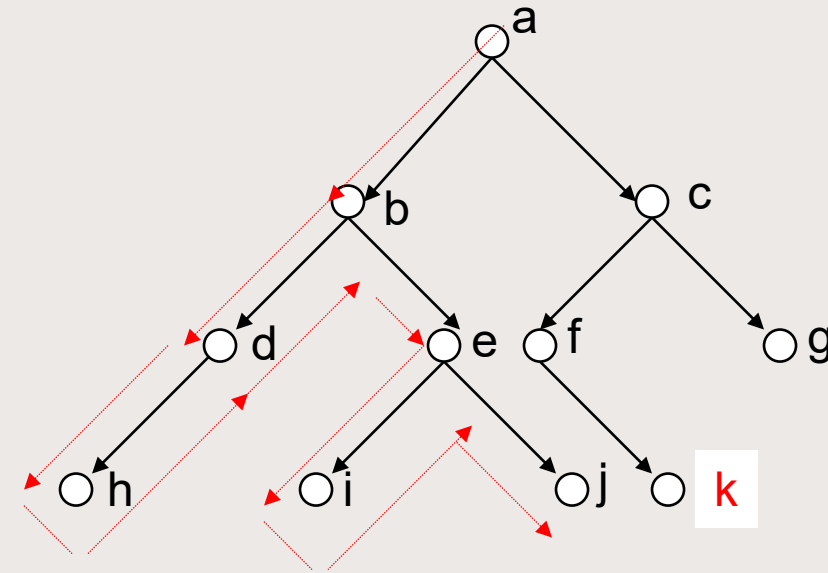


ESTRATEGIAS BÁSICAS DE LOS MÉTODOS DE BÚSQUEDA NO INFORMADAS

- Búsqueda en Profundidad
- Búsqueda en Amplitud
- Búsqueda de Costo Uniforme
- Algoritmo Dijkstra

BÚSQUEDA EN PROFUNDIDAD

- Recorre sistemáticamente una estructura de árbol para encontrar el nodo solución, recorriéndolo en profundidad.
- La Búsqueda es conducida a partir del estado inicial recorriendo tantos nodos cuanto fuere necesario para encontrar el estado objetivo.
- Nodo escogido: Siempre el mas distante del nodo inicial
- Implementación: Cola = LIFO
- Esta estrategia puede ser ineficiente ya que el algoritmo puede dedicarse a recorrer un camino demasiado largo que no conduzca a ninguna solución o meterse en bucles infinitos.



BÚSQUEDA EN PROFUNDIDAD - ALGORITMO

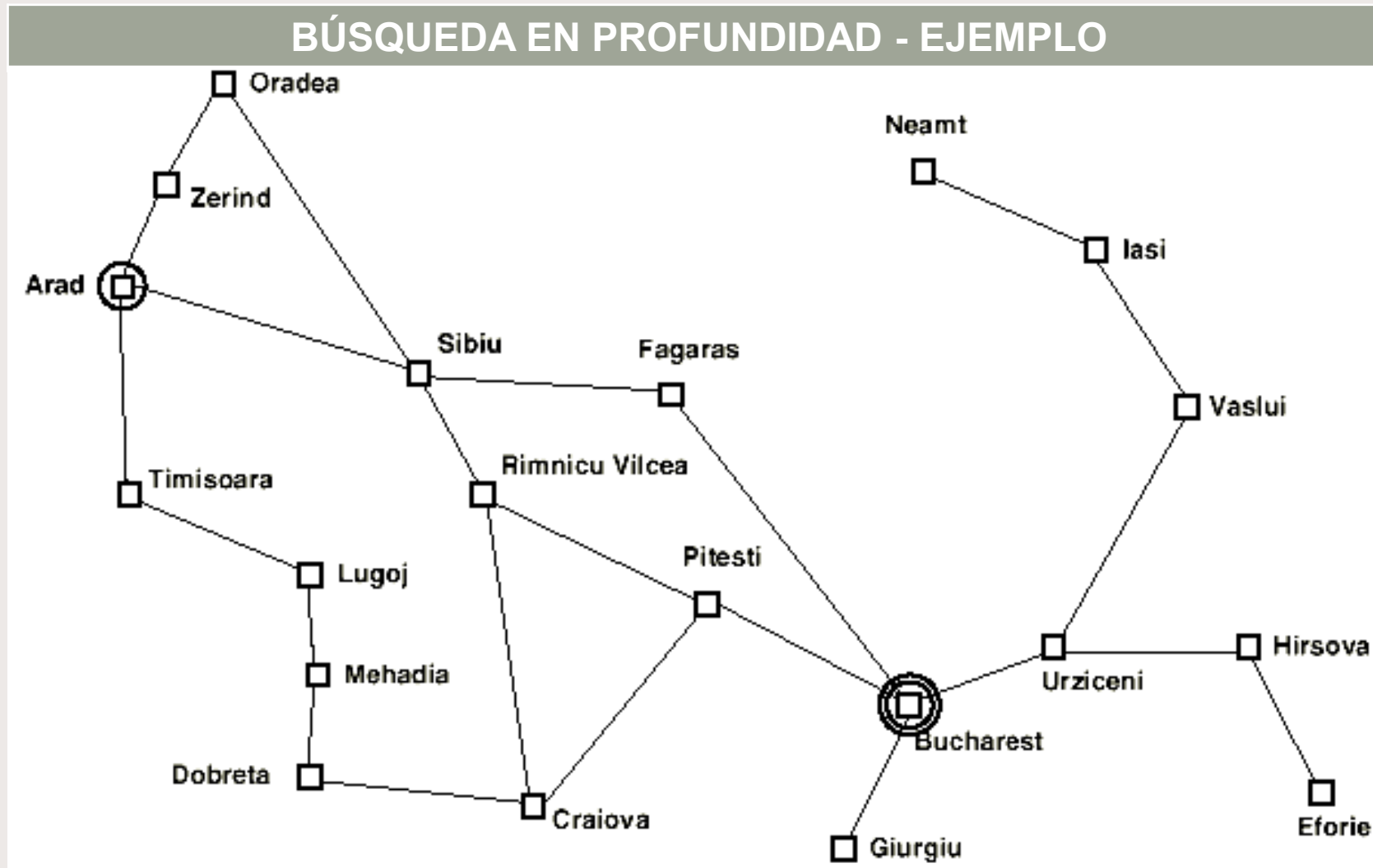
Crea una lista de nodos inicialmente con el nodo raíz (estado inicial).
Esta lista se maneja como una pila LIFO.

Lazo:

1. Si la lista está vacía, salir.
2. Extraer el primer nodo de la lista
3. Si la lista contiene el nodo meta, salir.

La solución es el primer nodo de la lista

4. Para cada hijo del nodo actual, expandirlo y añadir todos sus hijos como nuevos nodos al principio de la cola.

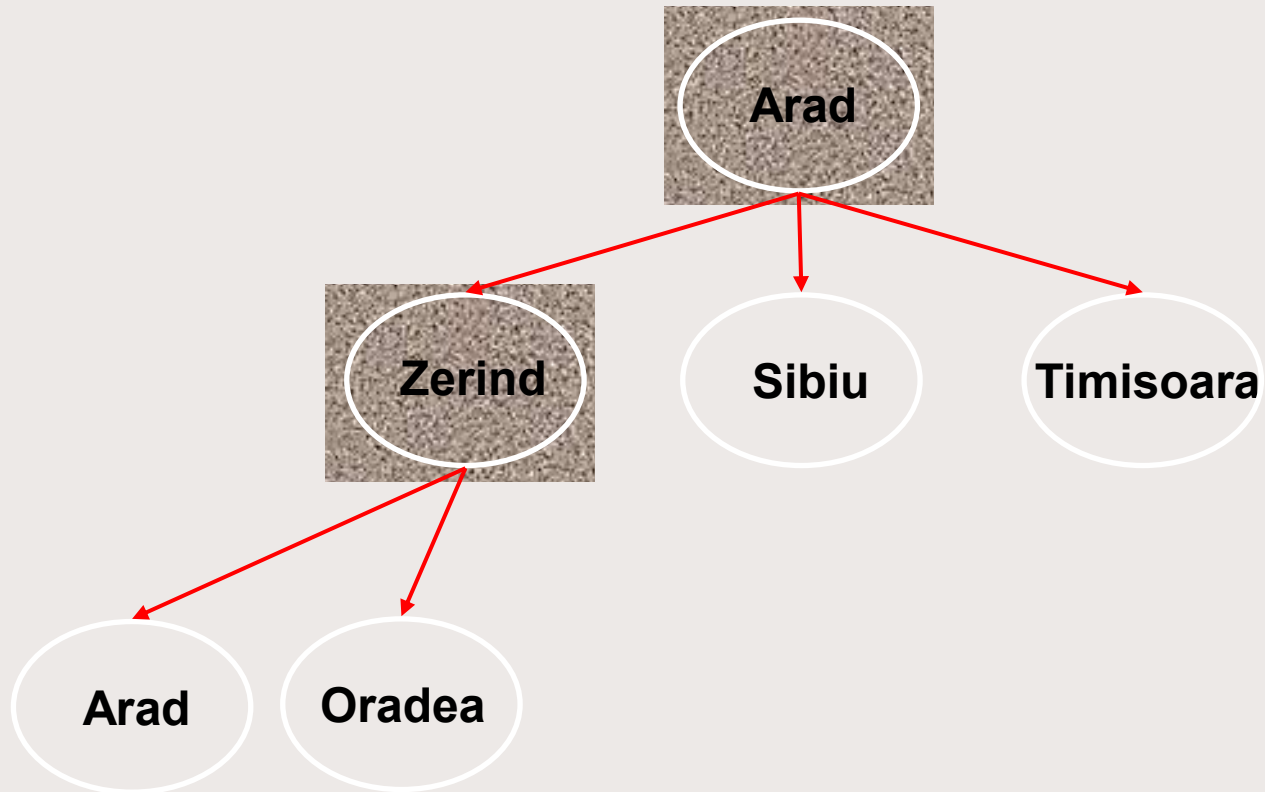


RESOLUCIÓN DE PROBLEMAS – BÚSQUEDA NO INFORMADA

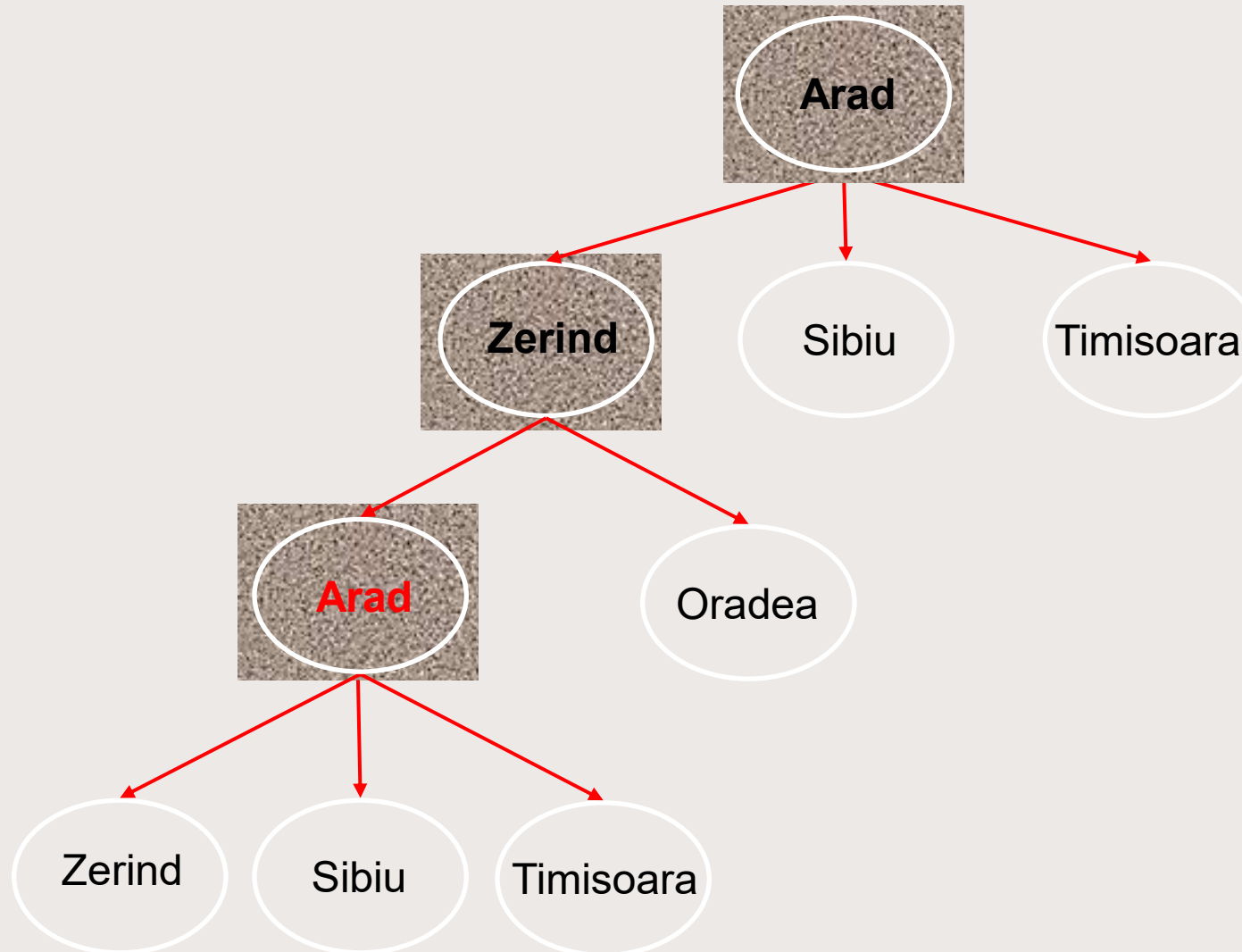


BÚSQUEDA EN PROFUNDIDAD - EJEMPLO

Siempre expande el nodo más profundo no expandido.



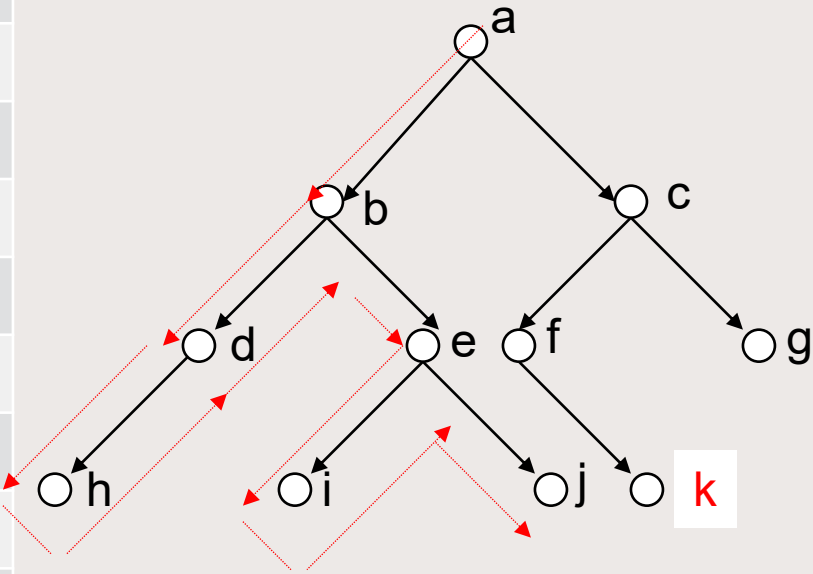
BÚSQUEDA EN PROFUNDIDAD - EJEMPLO





BÚSQUEDA EN PROFUNDIDAD - ESTADO DE COLA LIFO

Estado de la cola											Iteración
a											
	b	c									Extraer a + hijos de a
		d	e	c							Extraer b + hijos de b
			h	e	c						Extraer d + hijos de d
				e	c						Extraer h
					i	j	c				Extraer e + hijos de e
						j	c				Extraer i
							c				Extraer j
								f	g		Extraer c + hijos de c
									k	g	Extraer f + hijos de f
El recorrido realizado sería: a b d h e i j c f k g											



BÚSQUEDA EN PROFUNDIDAD – DESEMPEÑO

Eficiente para árboles de búsqueda con muchos niveles. Se torna ineficiente si hubiere una gran cantidad de ramas (b).

Completa?: No. Falla en espacios de profundidad infinita o con bucles. Es necesario modificar para evitar estados repetidos en el camino. Es completa en espacios finitos.

Tiempo?: $O(b^m)$ es malo si $m \gg d$.

Espacio. $O(bm)$. Esto si el espacio es lineal.

Optima?: No.

Medidas de rendimiento

Compleitud: ¿está garantizado que el algoritmo encuentre una solución cuando esta exista?

Optimización: ¿encuentra la estrategia la solución óptima?

Complejidad en tiempo: ¿cuánto tarda en encontrar una solución?

Complejidad en espacio: ¿cuánta memoria se necesita para el funcionamiento de la búsqueda?

Parámetros de complejidad:

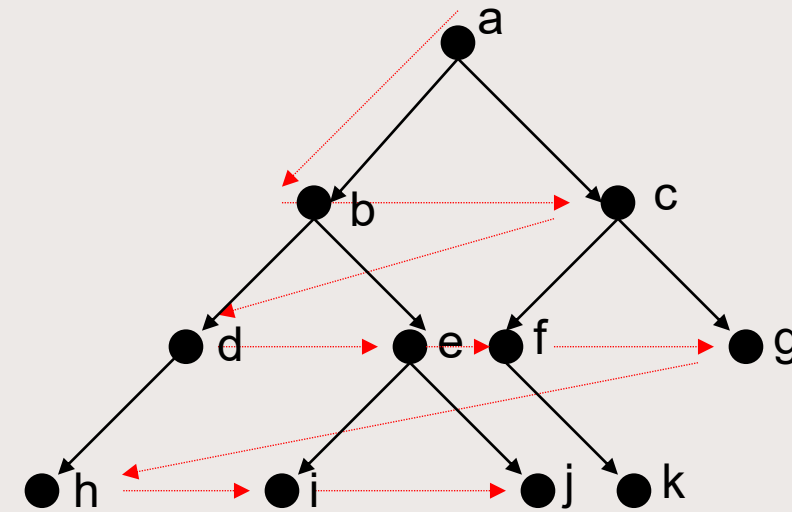
b – número máximo de sucesores de cualquier nodo

d – profundidad de la solución de menor costo

m – profundidad máxima del espacio de estado.

BÚSQUEDA EN AMPLITUD

- El algoritmo requiere que se expandan todos los nodos a una profundidad en el árbol de búsqueda antes de expandir cualquier nodo del próximo nivel.
- Los nodos superficiales se expanden antes que los más profundos.



BÚSQUEDA EN AMPLITUD - ALGORITMO

Crear una lista de nodos e inicializarla con el nodo raíz (estado inicial). Esta lista se maneja como una cola FIFO.

Lazo

1. Si la lista está vacía, terminar
2. Quitar el primer nodo de la lista
3. Si la lista contiene el nodo meta, salir.

La solución es el primer nodo de la lista

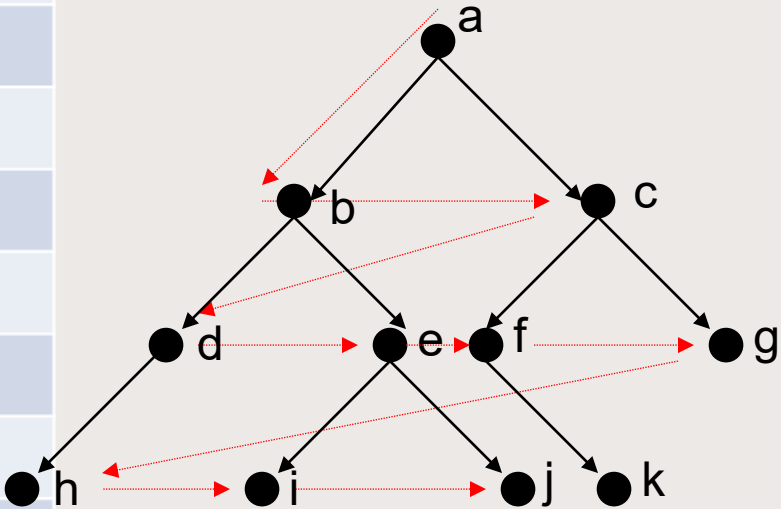
4. Expandir el nodo actual y añadir todos sus hijos como nuevos nodos al final de la cola.

RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA

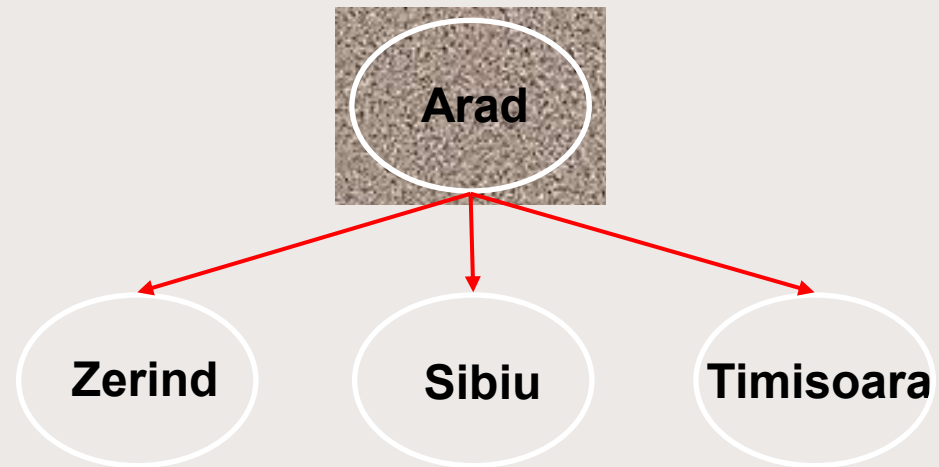


BÚSQUEDA EN AMPLITUD - ESTADO DE COLA FIFO

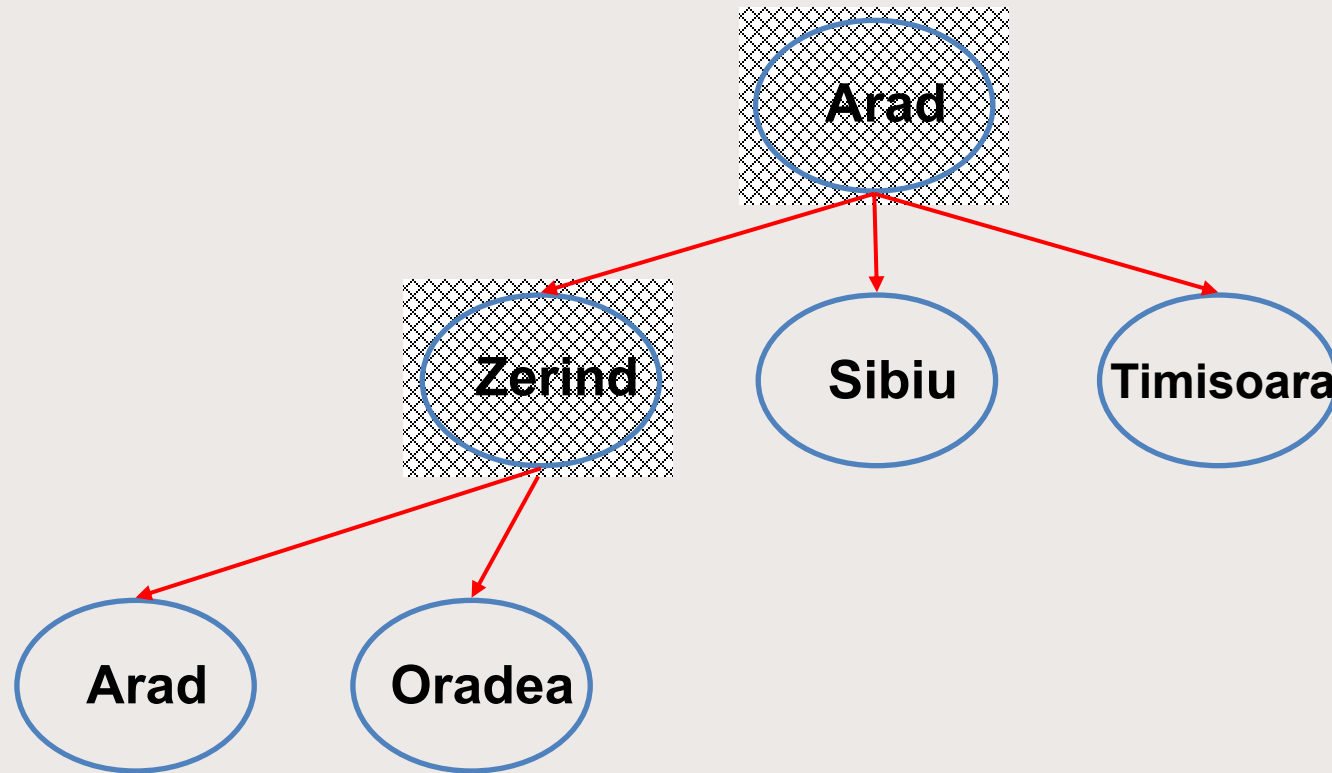
Estado de la cola											Iteración
a											
	b	c									Extraer a + hijos de a
		c	d	e							Extraer b + hijos de b
			d	e	f	g					Extraer d + hijos de d
				e	f	g	h				Extraer h
					f	g	h	i	j		Extraer e + hijos de e
						g	h	i	j	k	Extraer i
							h	i	j	k	Extraer j
								i	j	k	Extraer c + hijos de c
									j	k	Extraer f + hijos de f
El recorrido realizado sería: a b c d e f g h i j											



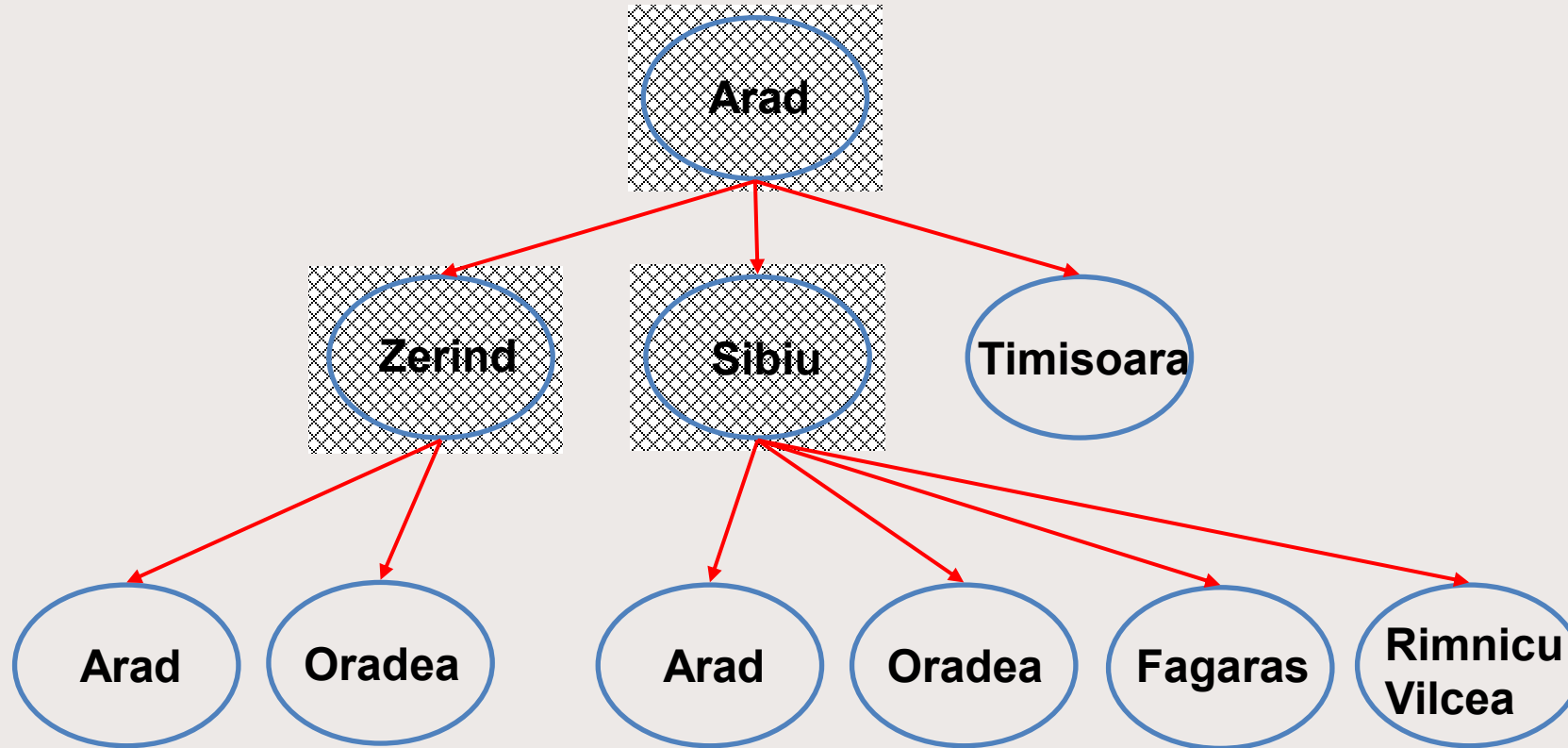
BÚSQUEDA EN AMPLITUD - EJEMPLO



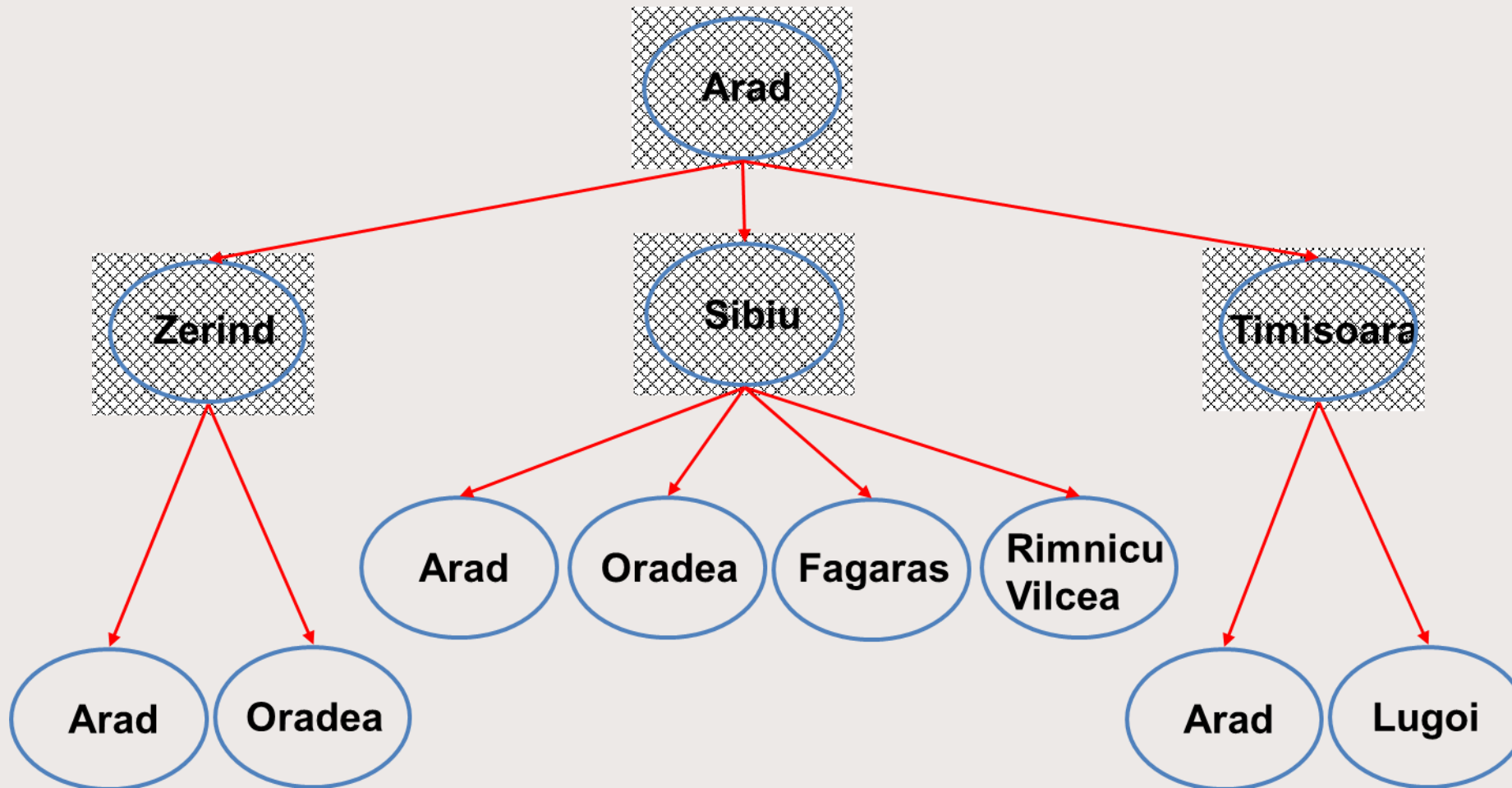
BÚSQUEDA EN AMPLITUD - EJEMPLO



BÚSQUEDA EN AMPLITUD - EJEMPLO



BÚSQUEDA EN AMPLITUD - EJEMPLO



BÚSQUEDA EN AMPLITUD – DESEMPEÑO

Proporciona un buen desempeño para problemas donde la solución puede ser encontrada después de ser recorrido un número pequeño de niveles.

Completa?: Si, si b es finito.

Tiempo: $1 + b + b^2 + \dots + b^d = O(b^d)$

Espacio: $O(b^d)$

Optima?: Si (si costo = 1 por paso). En general no es óptima.

Medidas de rendimiento

Compleitud: ¿está garantizado que el algoritmo encuentre una solución cuando esta exista?

Optimización: ¿encuentra la estrategia la solución óptima?

Complejidad en tiempo: ¿cuánto tarda en encontrar una solución?

Complejidad en espacio: ¿cuánta memoria se necesita para el funcionamiento de la búsqueda?

Parámetros de complejidad:

b – número máximo de sucesores de cualquier nodo

d – profundidad de la solución de menor costo

m – profundidad máxima del espacio de estado.

BÚSQUEDA DE COSTO UNIFORME

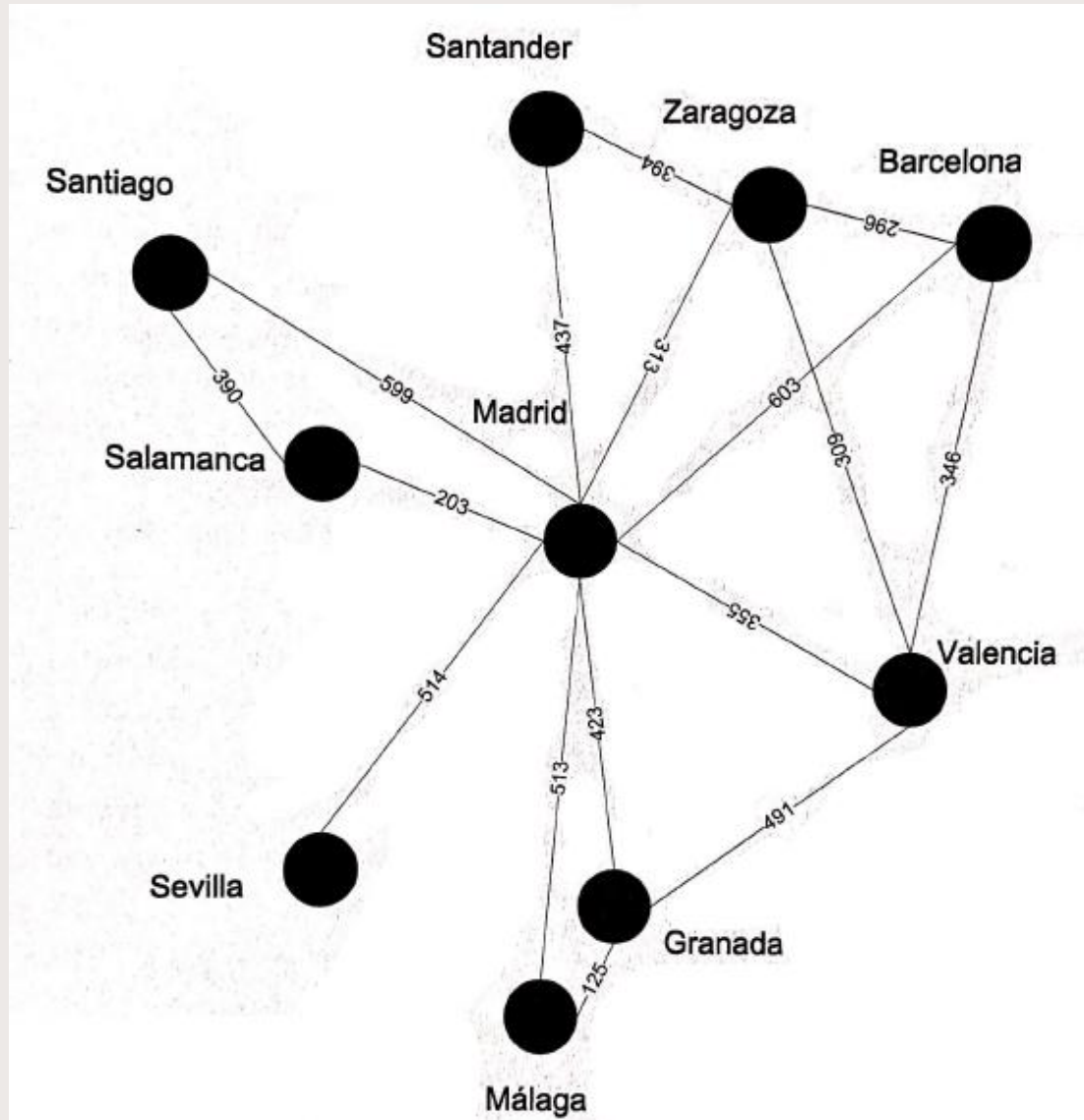
Realiza la búsqueda en el espacio de estados teniendo en cuenta el costo del camino recorrido desde el nodo raíz al nodo evaluado.

Utiliza una cola con prioridad para almacenar los nodos frontera.

La cola con prioridad se ordena según el costo de cada nodo.

Es óptima, completa siempre que la profundidad del árbol sea finita pero la complejidad no es tan obvia.

RESOLUCIÓN DE PROBLEMAS – BÚSQUEDA DE COSTO UNIFORME



Paso 1.

Nodos frontera: **Málaga (0)**

Seleccionamos: Málaga (0)

Hijos no visitados: Madrid(513), Granada(125)

Paso 2.

Nodos frontera: **Granada(125), Madrid(513)**

Seleccionamos: Granada(125)

Hijos no visitados: Madrid(548), Valencia(616)

Paso 3.

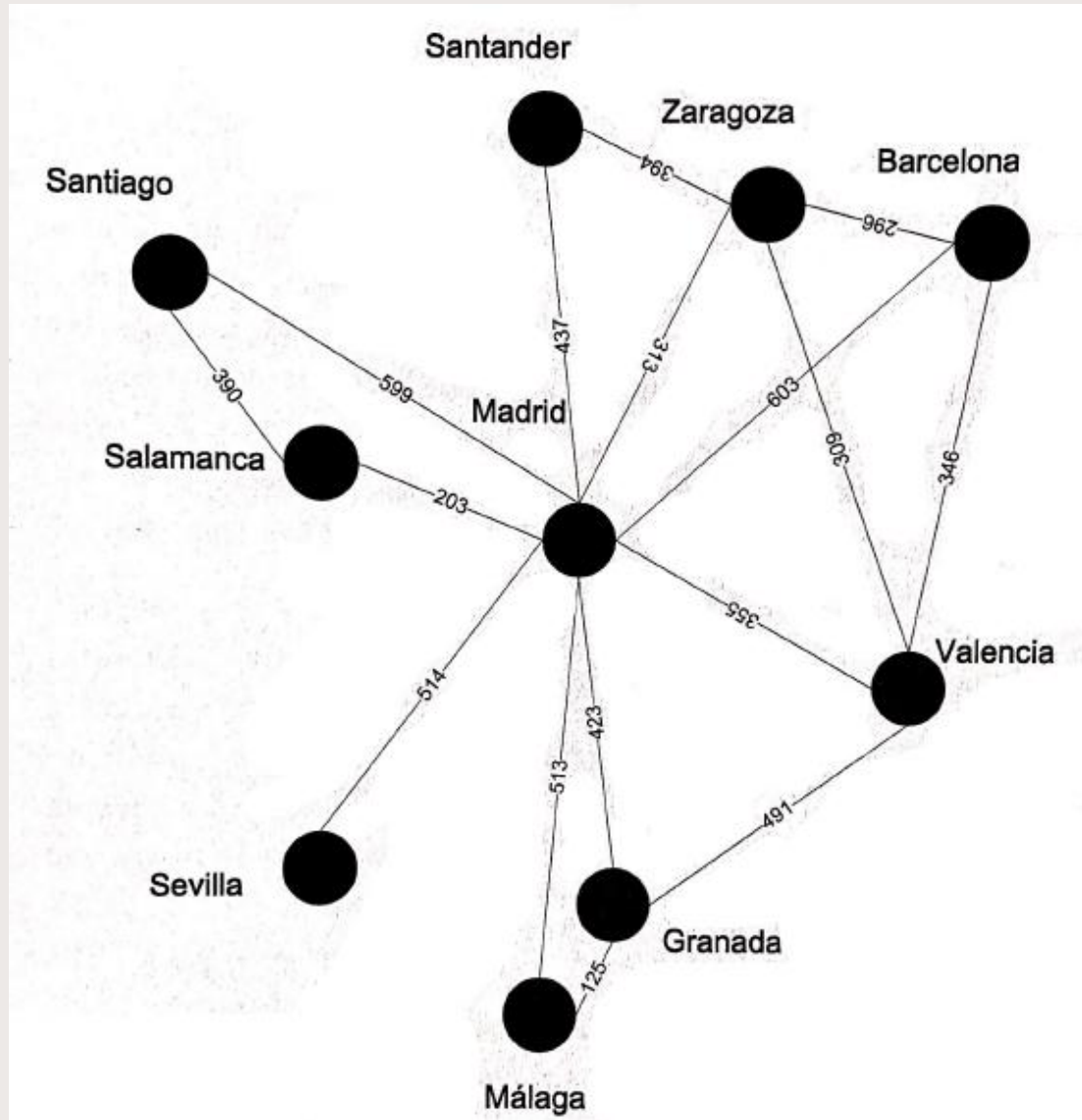
Nodos frontera: Madrid(513), **Valencia(616)**

Seleccionamos: Madrid(513)

Hijos no visitados: Valencia(868), Barcelona(1116), Zaragoza(826), Santander(950), Santiago(1112), Salamanca(716), Sevilla(1027)

* En Negrita: Nodos que se añaden nuevos o que cambian su valor $g(n)$.

RESOLUCIÓN DE PROBLEMAS – BÚSQUEDA DE COSTO UNIFORME



Paso 4.

Nodos frontera: Valencia(616), **Salamanca(716)**, **Zaragoza(826)**, **Santander(950)**, **Santiago(1112)**, **Barcelona(1116)**, **Sevilla(1027)**

Seleccionamos: Valencia(616)

Hijos no visitados: Barcelona(962), Zaragoza(925)

Paso 5.

Nodos frontera: Salamanca(716), Zaragoza(826), Santander(950), **Barcelona(962)**, Santiago(1112), Sevilla(1027)

Seleccionamos: Salamanca(716)

Hijos no visitados: Santiago(1106)

Paso 6.

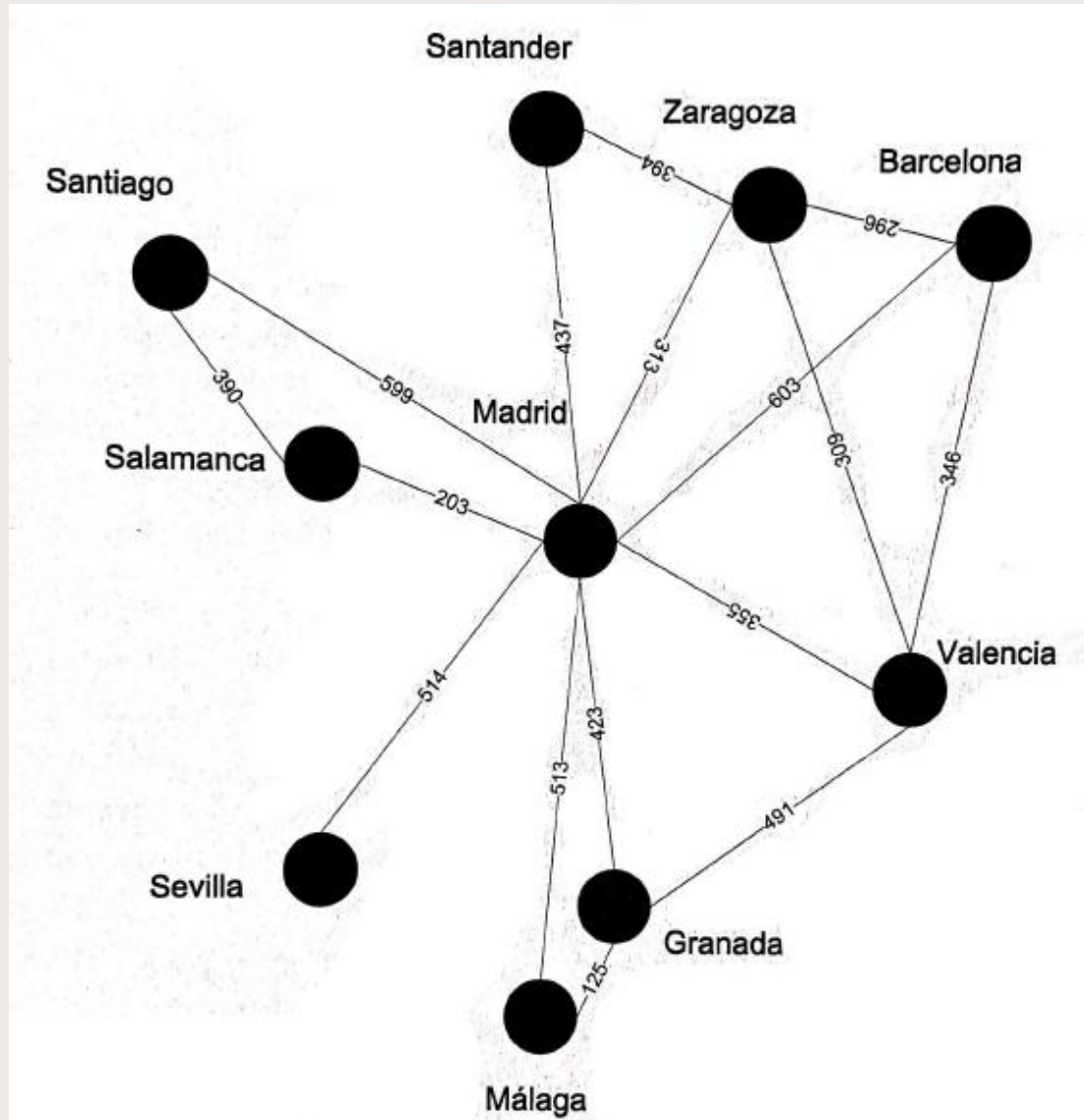
Nodos frontera: Zaragoza(826), Santander(950), Barcelona(962), **Santiago(1106)**, Sevilla(1027)

Seleccionamos: Zaragoza(826)

Hijos no visitados: Barcelona(1122), Santander(1220)

* En Negrita: Nodos que se añaden nuevos o que cambian su valor $g(n)$.

RESOLUCIÓN DE PROBLEMAS – BÚSQUEDA DE COSTO UNIFORME



Paso 7.

Nodos frontera: Santander(950), Barcelona(962), Santiago(1106), Sevilla(1027)

Seleccionamos: Santander(950)

Hijos no visitados: Ninguno

Paso 8.

Nodos frontera: Barcelona(962), Santiago(1106), Sevilla(1027)

Seleccionamos: Barcelona(962)

Hijos no visitados: Ninguno

Paso 9.

Nodos frontera: Santiago(1106), Sevilla(1027)

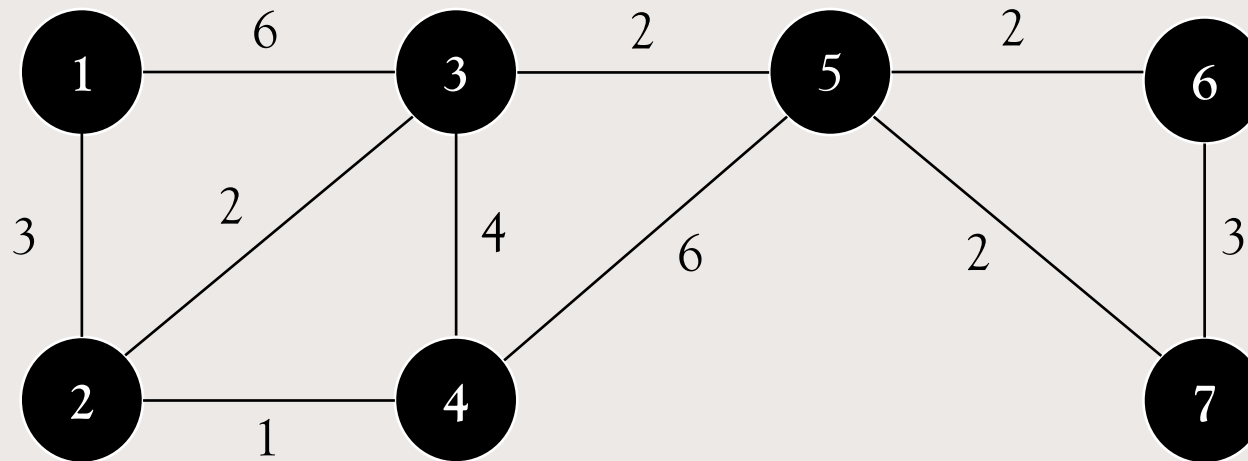
Seleccionamos: Santiago(1106)

* En Negrita: Nodos que se añaden nuevos o que cambian su valor $g(n)$.

ALGORITMO DIJKSTRA

1. Seleccionamos el nodo no visitado con menor distancia acumulada
2. Sumamos la distancia acumulada con la distancia a los nodos adyacentes y los etiquetamos con **[distancia_acumulada, padre]**. En caso de que alguno de los nodos adyacentes esté ya etiquetado, nos quedamos con el de menor distancia acumulada.
3. Marcamos el nodo actual como visitado y regresamos al paso 1.

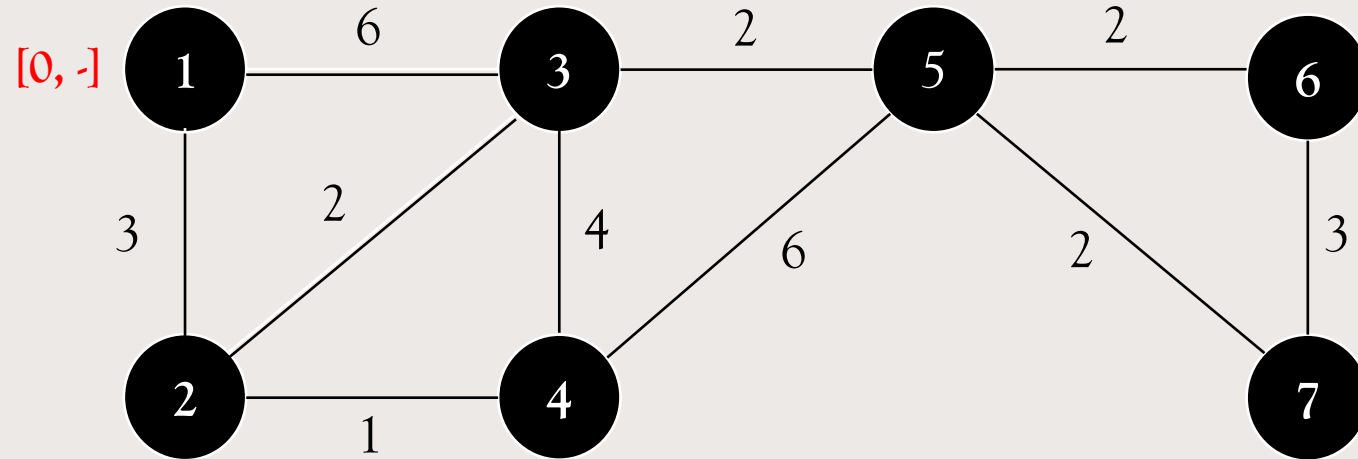
Ejemplo



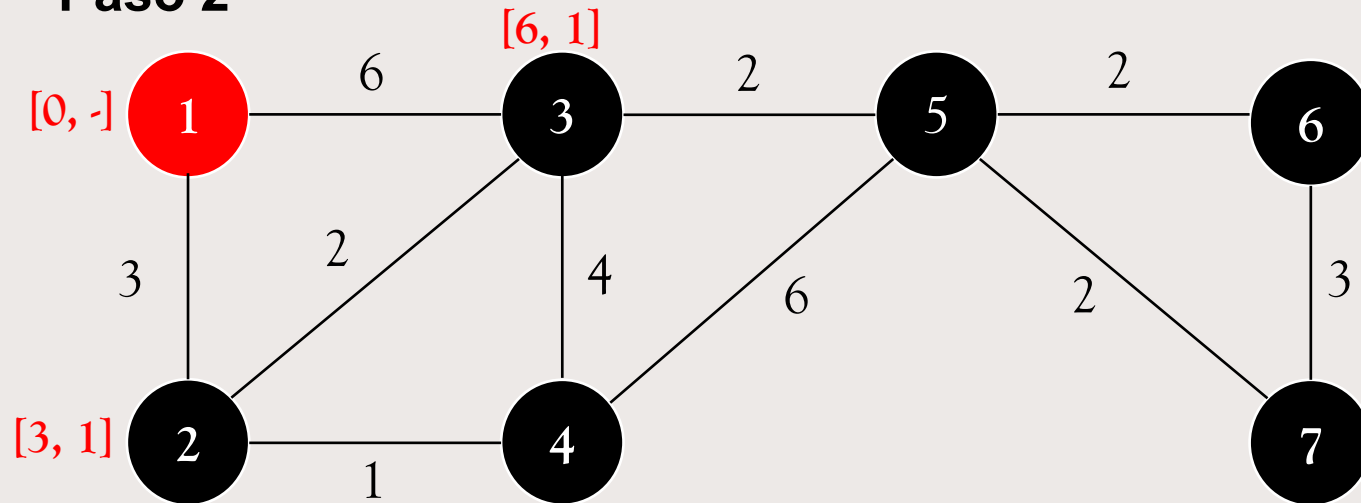
RESOLUCIÓN DE PROBLEMAS – ALGORITMO DIJKSTRA



Paso 1



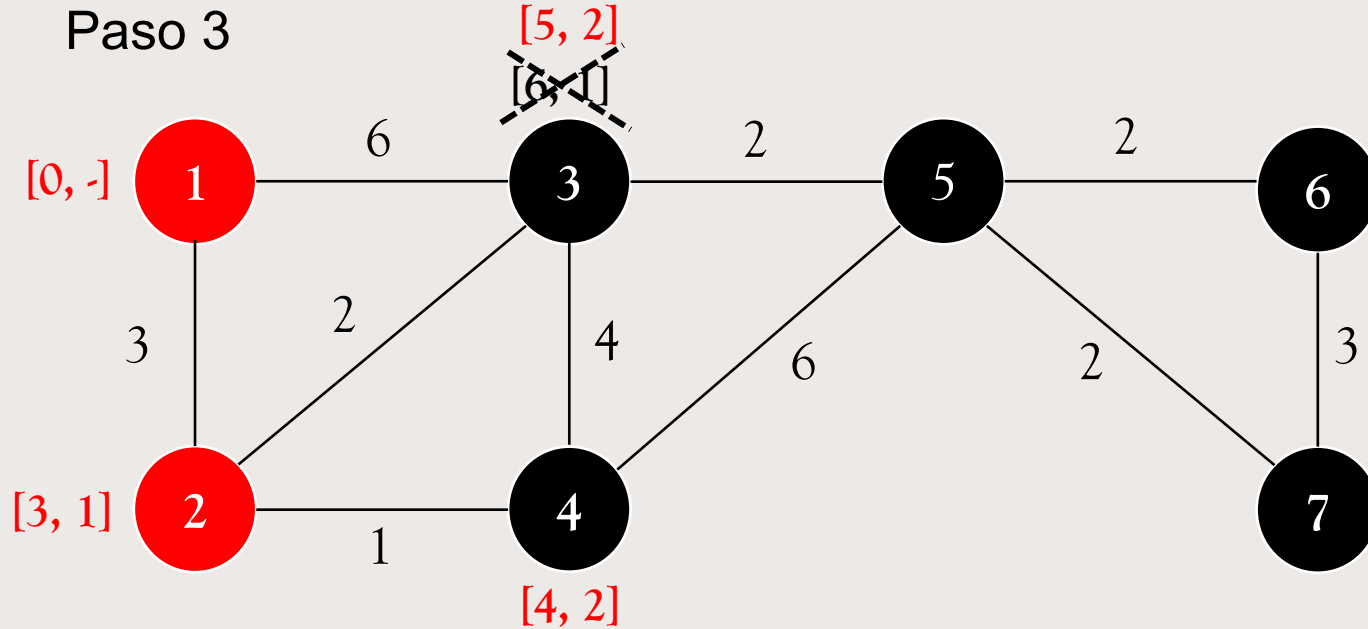
Paso 2



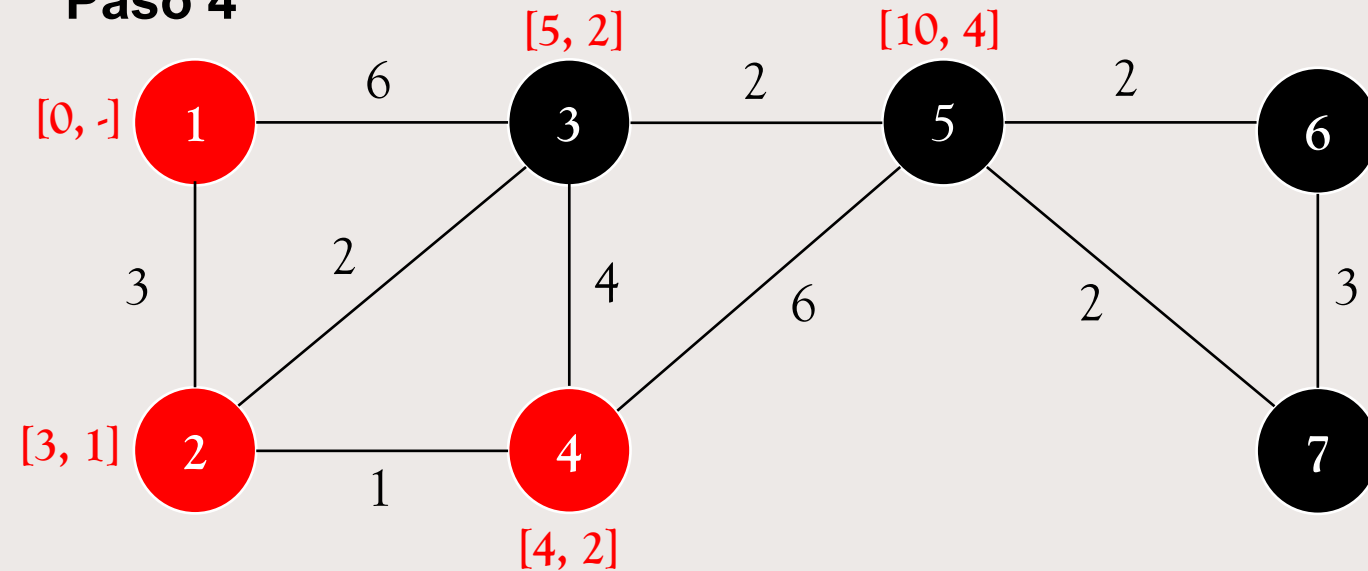
RESOLUCIÓN DE PROBLEMAS – ALGORITMO DIJKSTRA



Paso 3



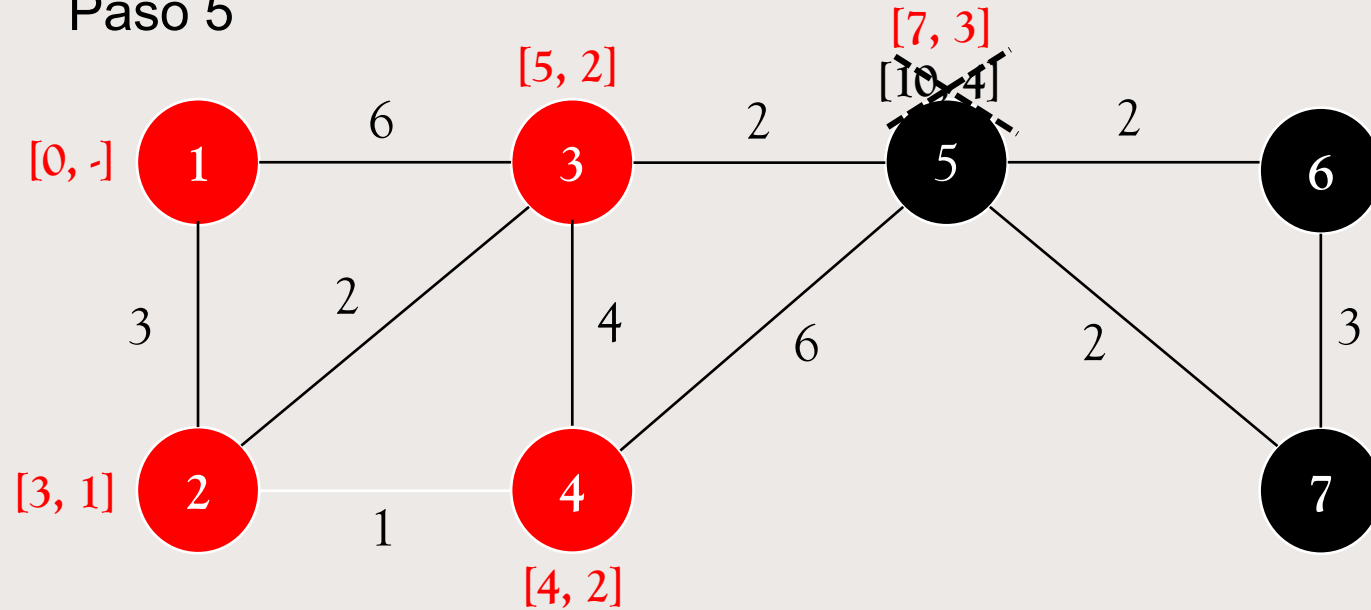
Paso 4



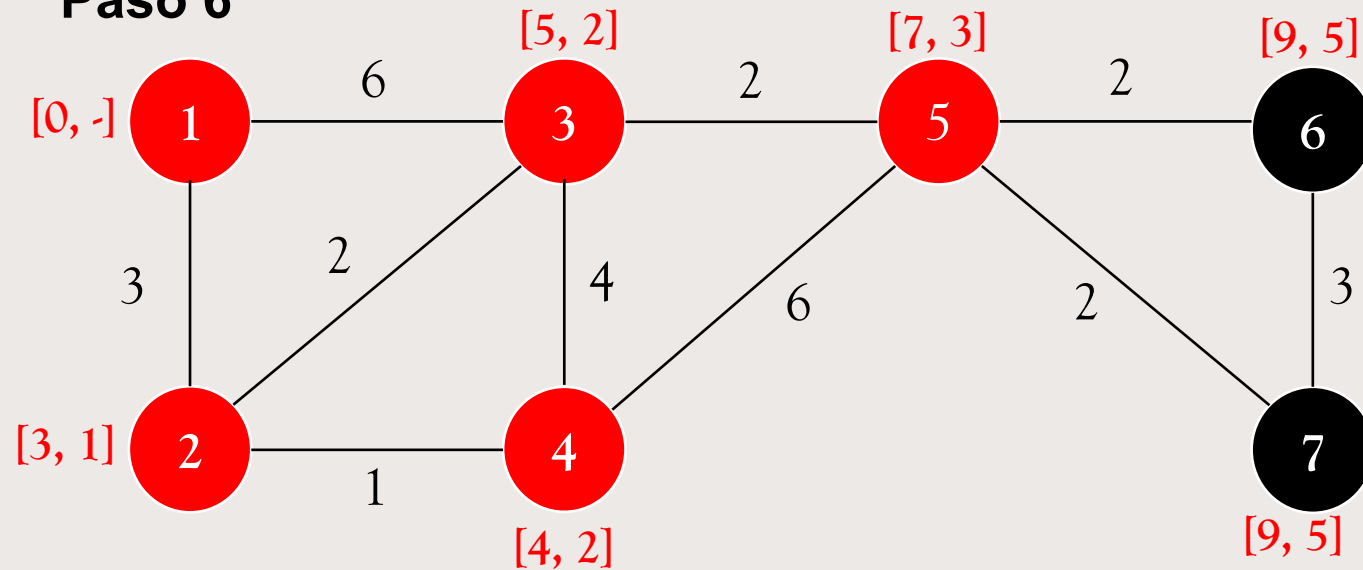
RESOLUCIÓN DE PROBLEMAS – ALGORITMO DIJKSTRA



Paso 5



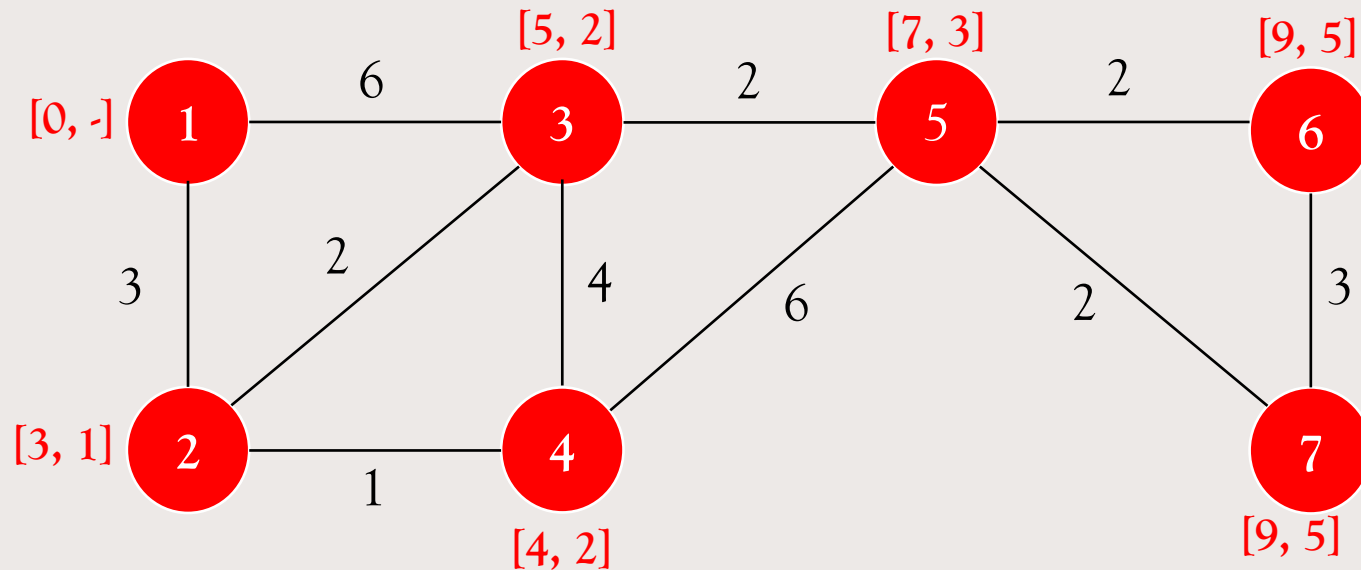
Paso 6



RESOLUCIÓN DE PROBLEMAS – ALGORITMO DIJKSTRA



Paso 7



Distancia o peso mínimo del nodo 1 al 7 = 9

Camino mínimo del nodo 1 al 7 = 1, 2, 3, 5, 7

MÉTODOS DE BÚSQUEDA INFORMADA (HEURÍSTICA)

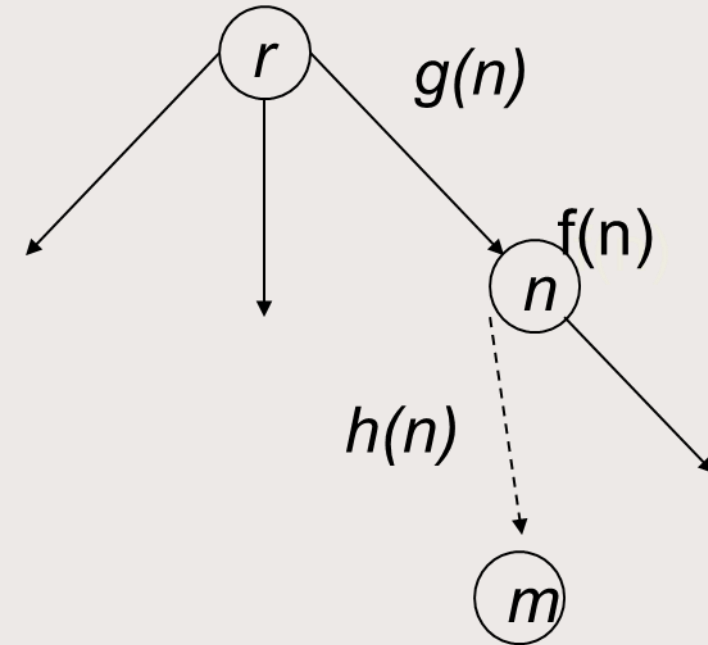
- Una estrategia de búsqueda mejorada para la resolución de problemas complejos.
- Heurística: “consejos” que determinan el sucesor de un estado dado.
- La búsqueda heurística utiliza información adicional sobre el problema específico como el coste o la distancia al estado final.

FUNCIONES HEURÍSTICAS

Función de Evaluación $f(n)$: Un estimador del menor costo en n .

Función Heurística $h(n)$: Un estimador del camino de menor costo desde n hasta el objetivo.

Función de costo $g(n)$: costo del nodo inicial hasta n .



$$f(n) = g(n) + h(n)$$

FUNCIONES DE EVALUACIÓN

Método para:

- Calcular un valor numérico para los estados sucesores de un estado dado.
- Decidir por el sucesor que tiene el menor valor.

Valores:

- Números no negativos
- El menor valor es el más prometedor

ESTRATEGIAS BÁSICAS DE LOS MÉTODOS DE BÚSQUEDA INFORMADA O HEURÍSTICA

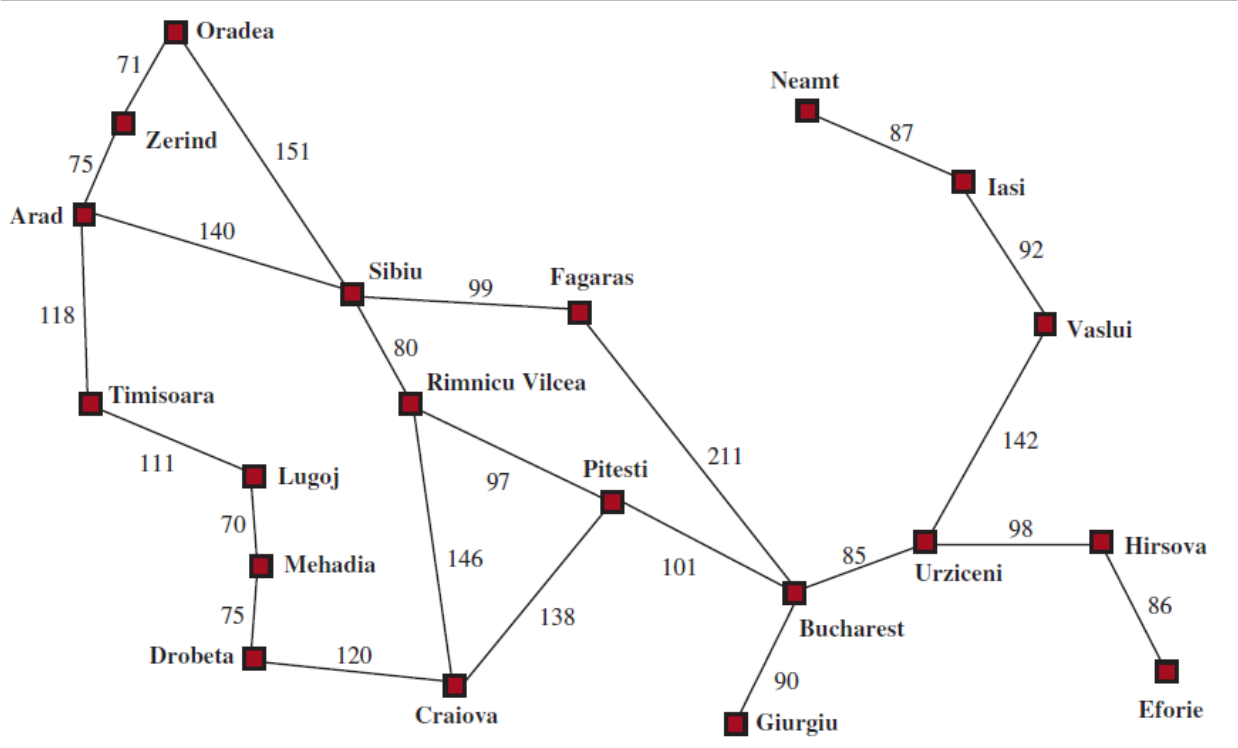
- Búsqueda Primero el Mejor
- Búsqueda A*

BÚSQUEDA PRIMERO EL MEJOR

- La idea básica es expandir el nodo que minimiza (maximiza) la función de evaluación $f(n) = h(n)$.
- Combina las ventajas de los métodos de búsqueda en amplitud y en profundidad, usando la función de evaluación $h(n)$
- Expande el nodo más prometedor (cercano al objetivo), conforme la función de evaluación $h(n)$.

BÚSQUEDA PRIMERO EL MEJOR

Mapa de Rumania con costos en Km.

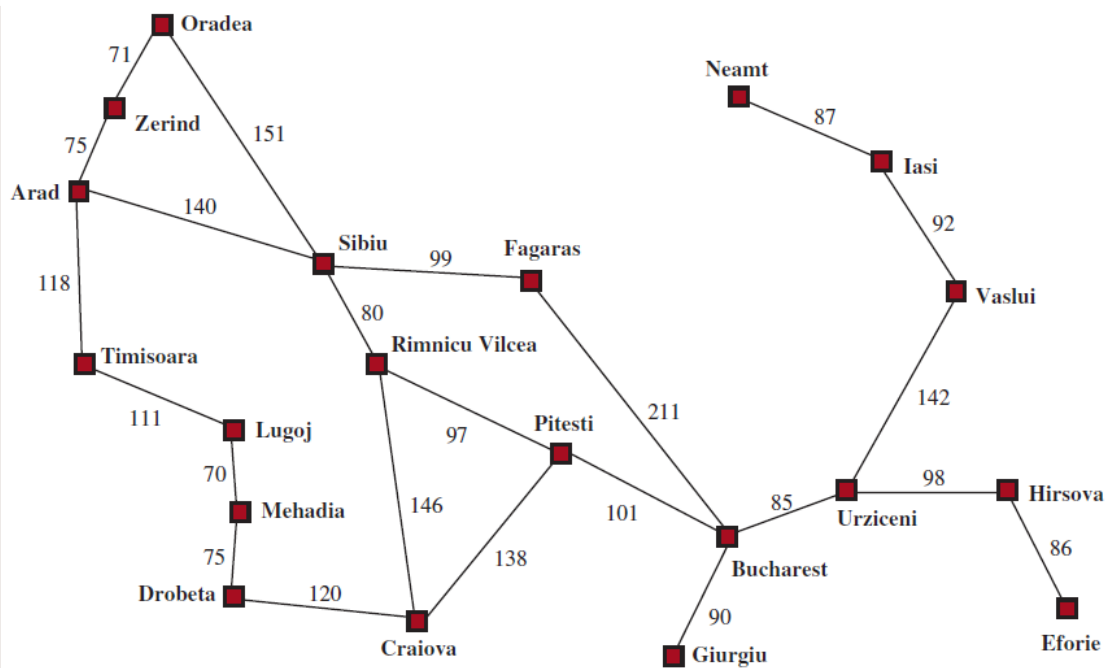


Valores de h_{DLR} . Distancia en línea recta a Bucarest

- $h(n)$ = estima el costo del camino del nodo n al objetivo
- $h(n) = 0$, si n es el nodo objetivo.
- $h(n)$ = distancia en línea recta del nodo n a Bucarest.

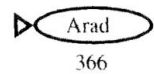
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

BÚSQUEDA PRIMERO EL MEJOR

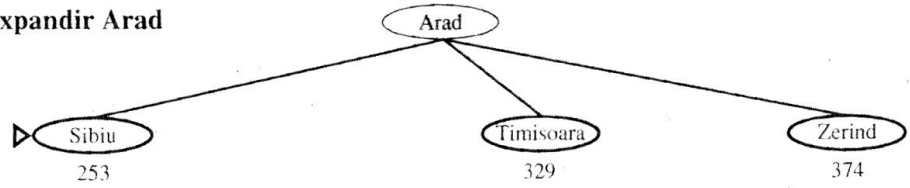


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

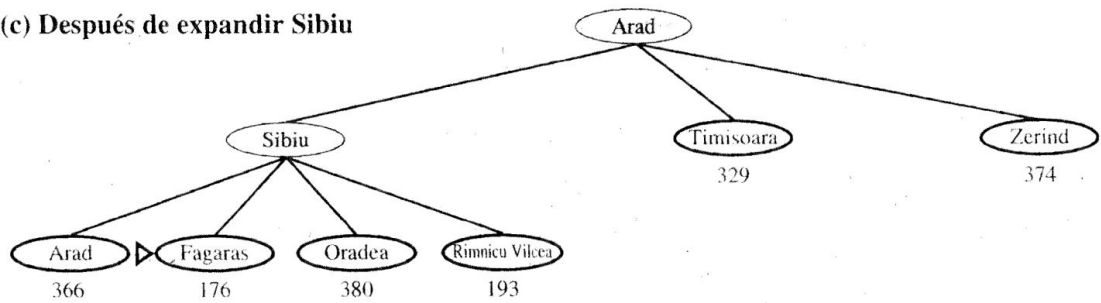
(a) Estado inicial



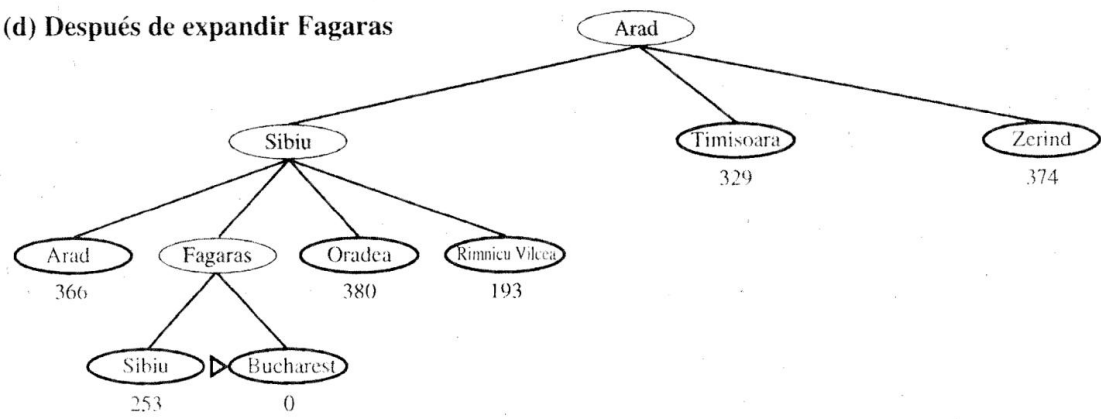
(b) Después de expandir Arad



(c) Después de expandir Sibiu



(d) Después de expandir Fagaras

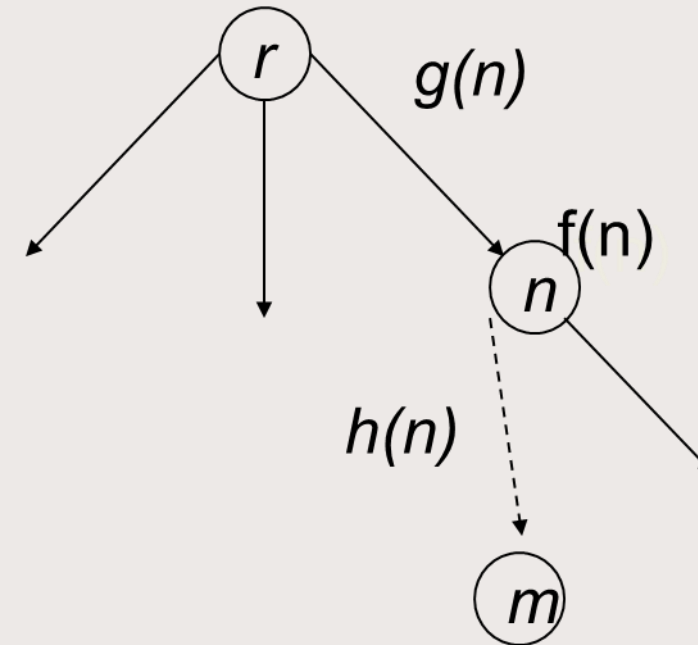


BÚSQUEDA A*

Utiliza la función heurística $h(n)$ y la función de costo $g(n)$:

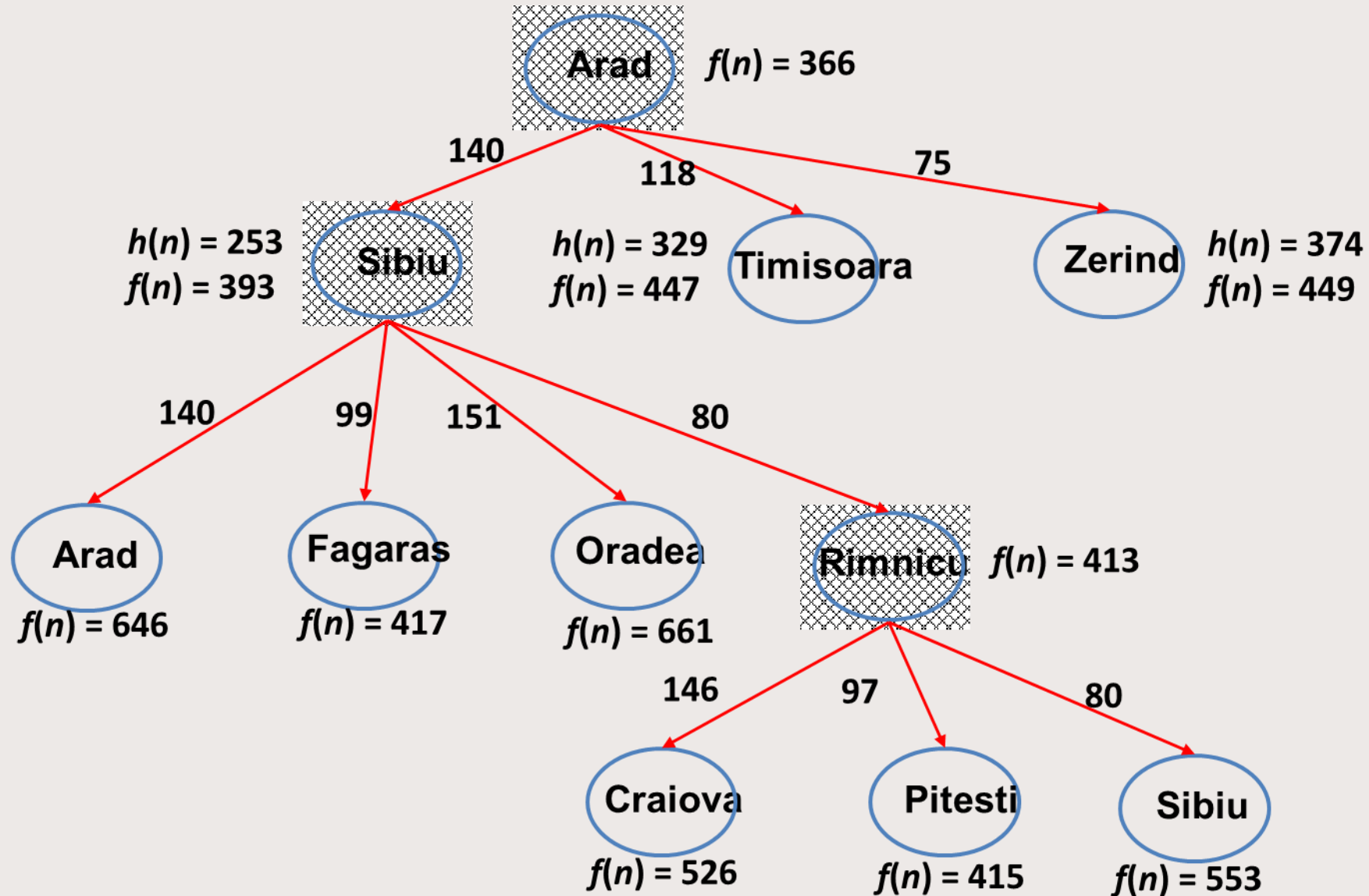
- $f(n) = h(n) + g(n)$
- $f(n)$ = Coste más barato estimado de la solución a través de n .

Así, si tratamos de encontrar la solución más barata, es razonable intentar primero el nodo con el valor más bajo de $g(n) + h(n)$.

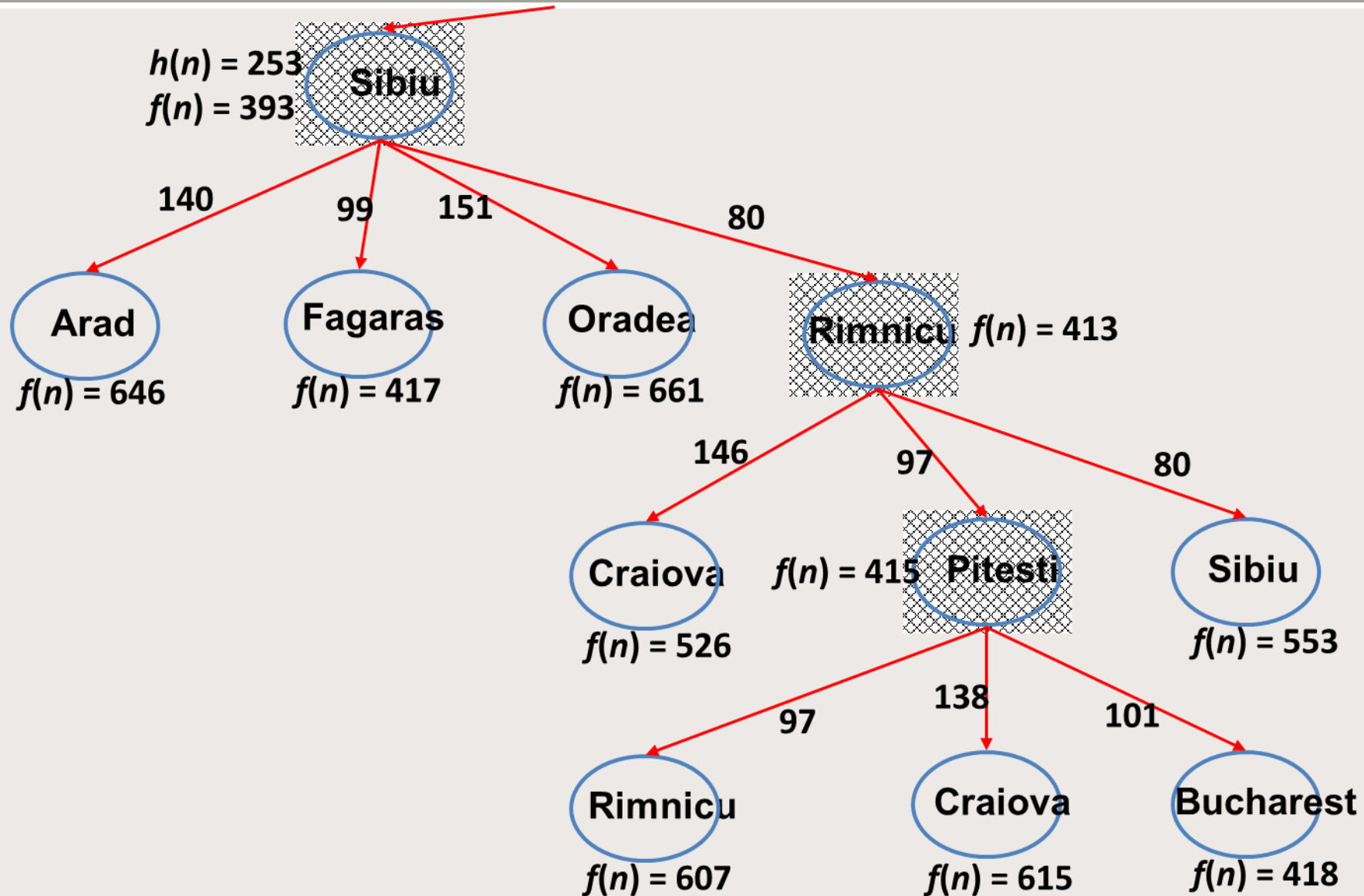


$$f(n) = g(n) + h(n)$$

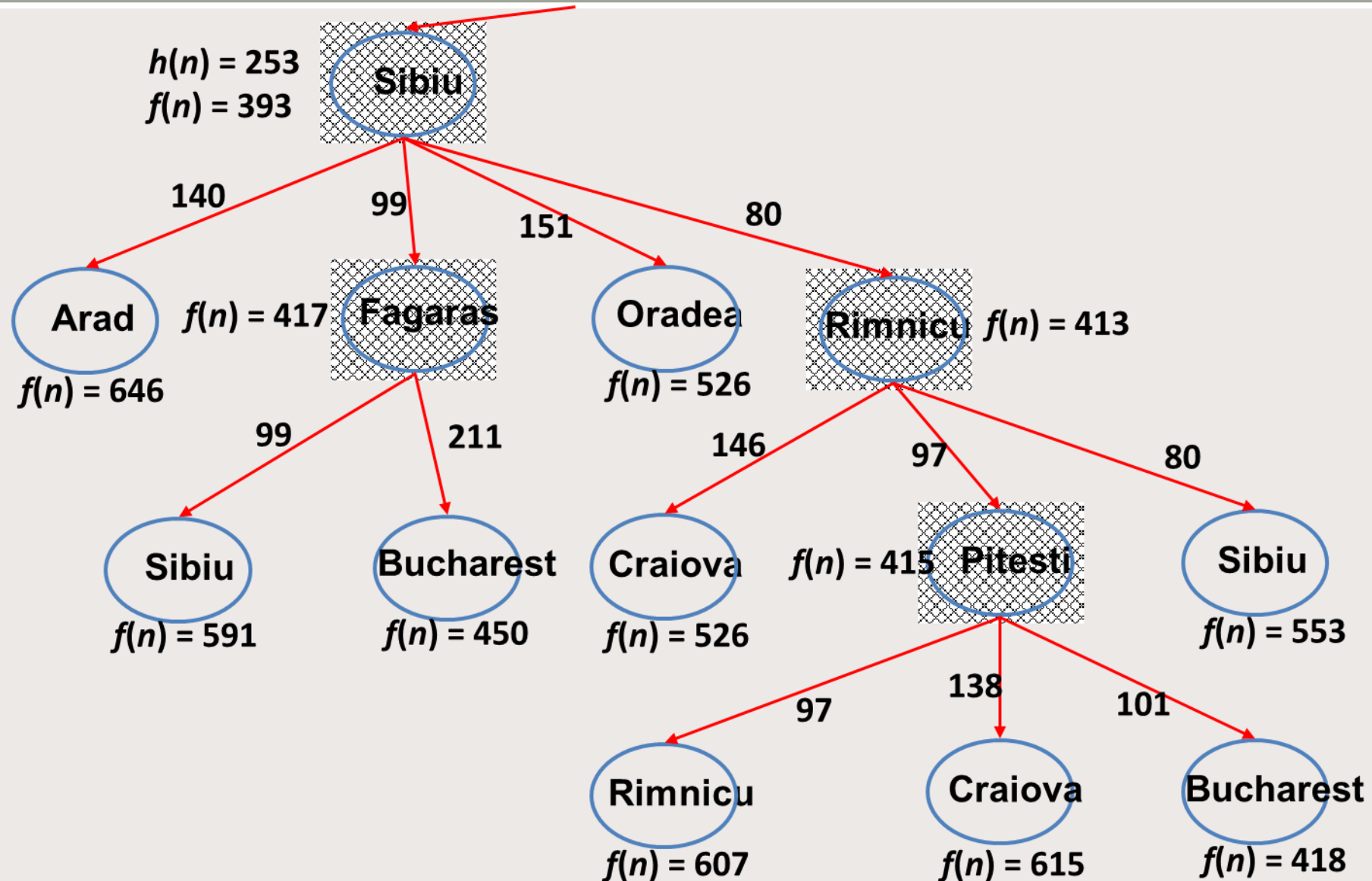
EJECUCIÓN DEL ALGORITMO DE BÚSQUEDA A*



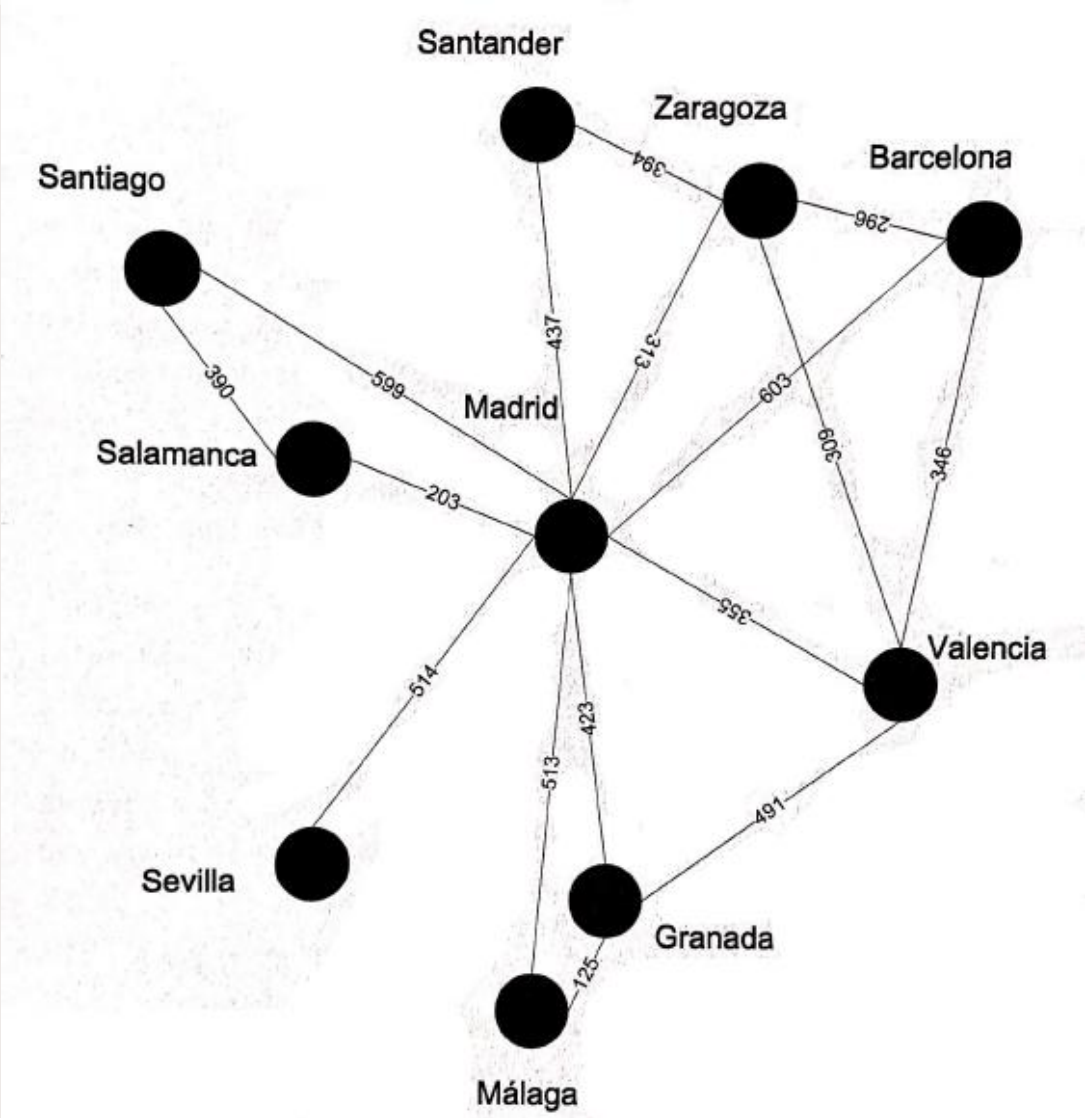
EJECUCIÓN DEL ALGORITMO DE BÚSQUEDA A*



EJECUCIÓN DEL ALGORITMO DE BÚSQUEDA A*



EJERCICIO. Encontrar el camino más corto desde Málaga hasta Santiago, aplicando los métodos primero el mejor y A*; y comparar los resultados.



Ciudad	Distancia en línea recta hasta Santiago
Zaragoza	658
Santiago	0
Santander	404
Málaga	763
Salamanca	326
Madrid	482
Valencia	771
Sevilla	633
Barcelona	876
Granada	737

BÚSQUEDA LOCAL

- La idea es comenzar con un estado inicial y mejorarlo iterativamente.
- Funciona con un solo estado actual y generalmente se mueve solo a los vecinos del estado.
- Típicamente, los caminos seguidos por la búsqueda no se retienen. Estos algoritmos almacenan apenas el estado actual (bajo uso de memoria) y no ven más allá de los vecinos inmediatos del estado.
- Pueden encontrar a menudo soluciones razonables en espacios de estados grandes o infinitos (continuos) para los cuales son inadecuados los algoritmos sistemáticos.

BÚSQUEDA LOCAL

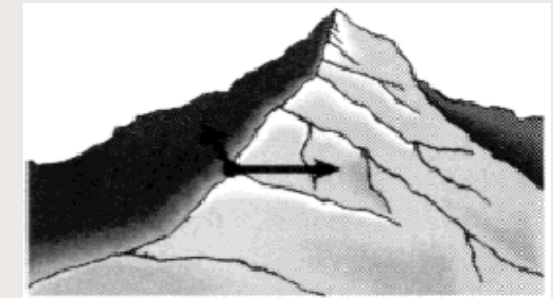
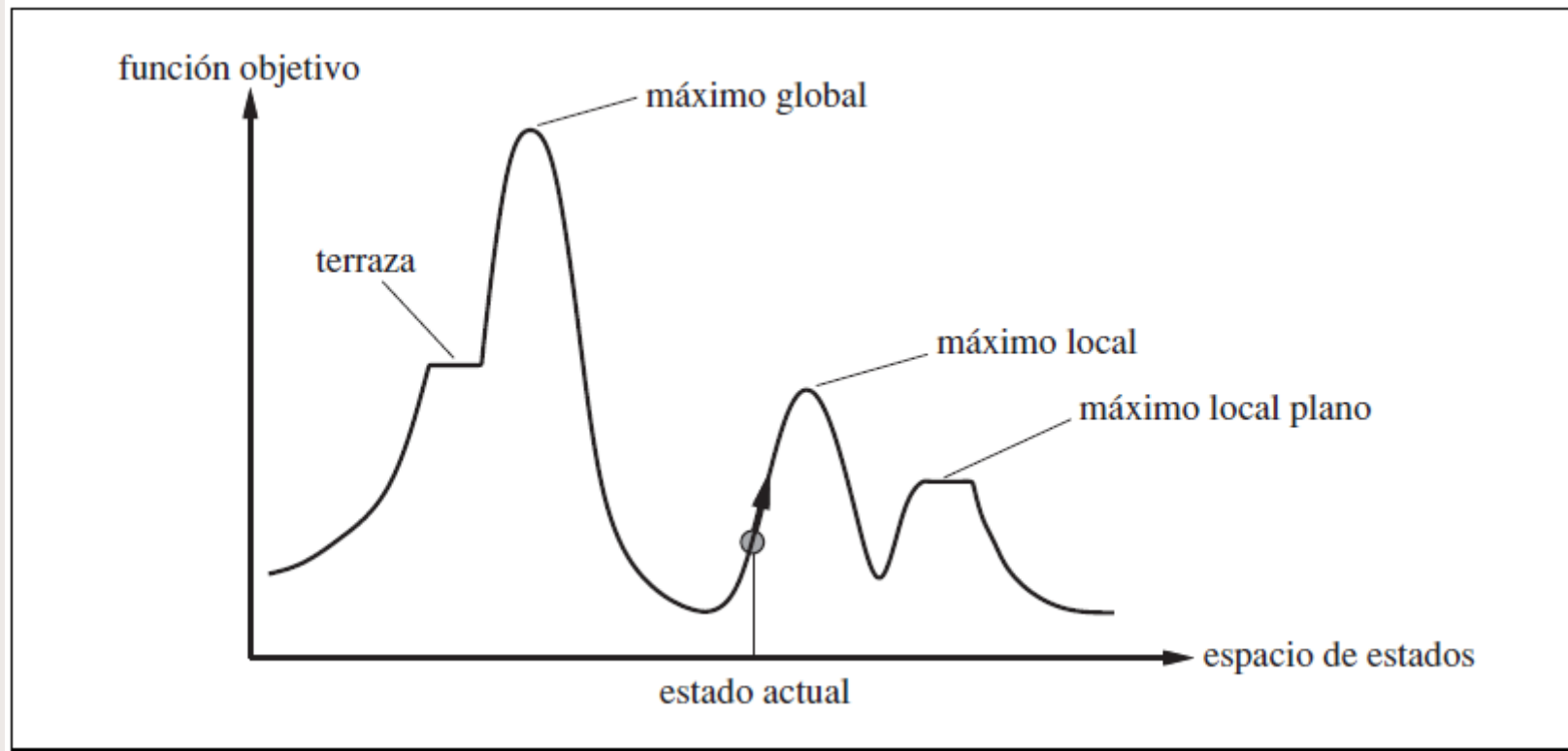
- Los estados (solución) pueden ser representados sobre una superficie (gráfico).
 - La altura en cualquier punto de la superficie corresponde a la función de evaluación del estado en ese punto.
- El algoritmo se “desplaza” por la superficie en búsqueda de puntos más altos.
 - El punto más alto (máximo global) corresponde a la solución óptima - el nodo donde la función de evaluación toma el valor máximo.

TIPOS DE BÚSQUEDA LOCAL

- **Hill Climbing.** Bucle que continuamente se mueve en dirección del valor creciente. Sólo realiza modificaciones que mejora el estado actual.
- **Temple simulado.** Puede realizar modificaciones que empeoran el estado del momento, para posiblemente mejorarlo en el futuro.
- **Haz local.** Mantiene k estados en vez de uno único.
- **Algoritmos Genéticos.** Es una búsqueda estocástica en la cual se sostiene una gran población de estados y son generados nuevos estados por mutación o cruce.

HILL CLIMBING (ASCENSIÓN DE COLINAS)

El objetivo es encontrar el máximo global modificando el estado actual para tratar de mejorarlo.



HILL CLIMBING (ASCENSIÓN DE COLINAS)

- El algoritmo mantiene una estructura de datos del nodo actual que necesita solo el registro del estado y su valor de función objetivo.
- No mira delante más allá de los vecinos inmediatos del nodo actual.
- Se parece a lo que hacemos cuando tratamos de encontrar la cumbre del Everest en una niebla mientras sufrimos amnesia.

Los siguientes problemas pueden ocurrir:

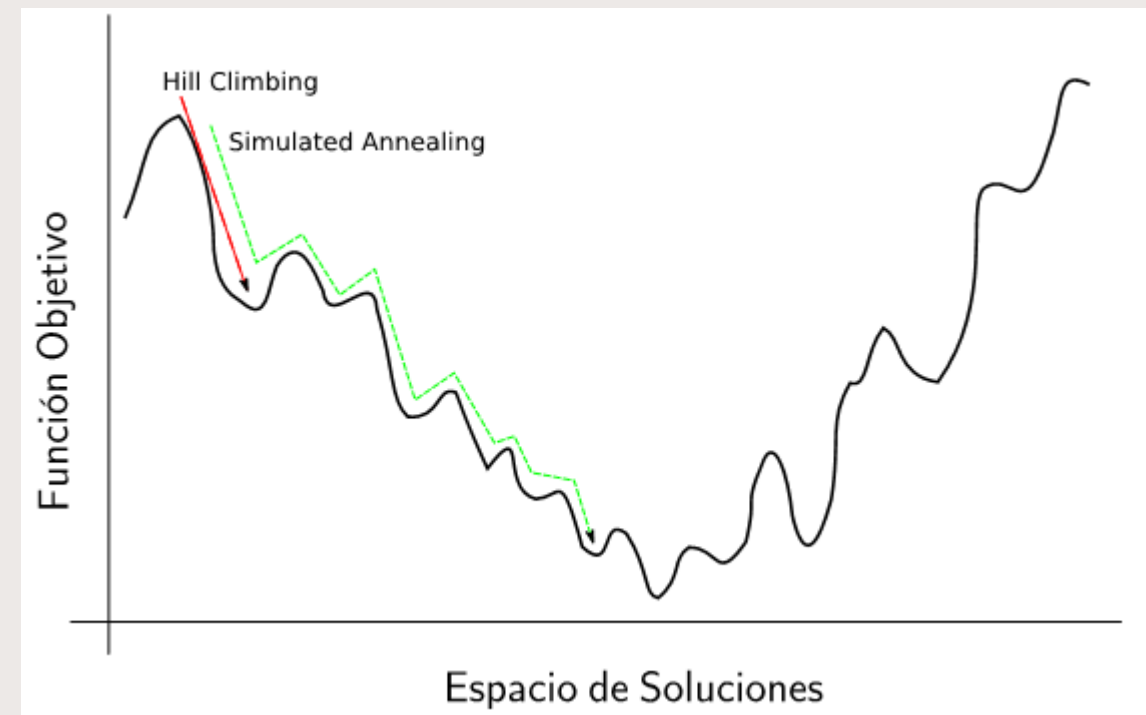
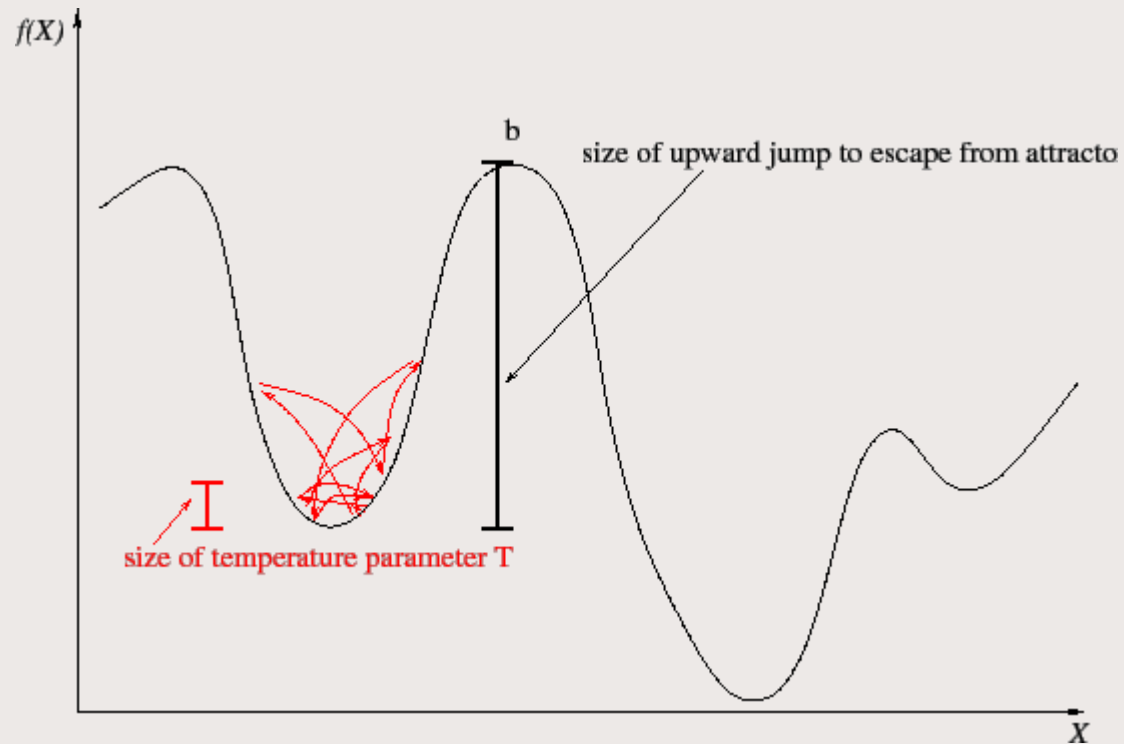
- Máximos locales: pueden ser confundidos con el máximo global, se atascarán y no podrán ir a ninguna otra parte.
- Crestas: causan una secuencia de máximos locales que hace difícil la navegación.
- Meseta: es un área del paisaje del espacio de estados donde la función de evaluación es plana y podría ser incapaz de encontrar su camino.

TEMPLE SIMULADO

- Es una analogía con el proceso utilizado para templar o endurecer metales y cristales, calentándolos a una temperatura alta y luego enfriarlos gradualmente.
- Similar a la Ascensión de Colinas pero con medios para escapar de los máximos locales.
- No siempre elegirá un vecino que mejore la solución actual, puede seleccionar una solución peor basado en una función probabilística.
- La función de probabilidad dependerá de (1) la diferencia relativa entre el estado actual y el nuevo y (2) un segundo factor denominado temperatura (T)

$$p = e^{(eval(E_{actual}) - eval(E_{nuevo})) / T}$$

TEMPLE SIMULADO



HAZ LOCAL

- El algoritmo de búsqueda por haz local guarda la pista de k estados.
- Comienza con estados generados aleatoriamente.
- En cada paso, se generan los sucesores de los k estados.
- Si alguno es un objetivo, paramos el algoritmo.
- Por otra parte, se seleccionan los k mejores sucesores de la lista completa y repetimos.

RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA

BOBLIOGRAFÍA

- INTELIGENCIA ARTIFICIAL.
Un enfoque moderno. Stuart
Russell y Peter Norvig.
- INTELIGENCIA ARTIFICIAL.
Fundamentos, práctica y
aplicaciones. Alberto García
Serrano